High-Order Tensor-Train Finite-Volume Methods for Shallow-Water Equations

M. ENGIN DANIS,[a] DUC P. TRUONG,[a] DEREK DESANTIS,[b] JAMIE LILLY,[b] MARK R. PETERSEN,[b] KIM Ø. RASMUSSEN,[a] AND BOIAN S. ALEXANDROV[a]

[a] *Theoretical Division, Los Alamos National Laboratory, Los Alamos, New Mexico*
[b] *Computer, Computational and Statistical Sciences Division, Los Alamos National Laboratory, Los Alamos, New Mexico*

(Manuscript received 9 August 2024, in final form 23 April 2025, accepted 7 May 2025)

ABSTRACT: In this paper, we introduce high-order tensor-train (TT) finite-volume methods for the shallow-water equations (SWEs). We present the implementation of the third-order upwind and the fifth-order upwind and weighted essentially nonoscillatory (WENO) reconstruction schemes in the TT format. It is shown in detail that the linear upwind schemes can be implemented by directly manipulating the TT cores, while the WENO scheme requires the use of TT cross interpolation for the nonlinear reconstruction. In the development of numerical fluxes, we directly compute the flux for the linear SWEs without using TT rounding or cross interpolation. For the nonlinear SWEs where the TT reciprocal of the shallow water layer thickness is needed for fluxes, we develop an approximation algorithm using Taylor series to compute the TT reciprocal. The performance of the TT finite-volume solver with linear and nonlinear reconstruction options is investigated under a physically relevant set of validation problems. In all test cases, the TT finite-volume method maintains the formal high-order accuracy of the corresponding traditional finite-volume method. In terms of speed, the TT solver achieves up to 124 times acceleration of the traditional full-tensor scheme.

KEYWORDS: Ocean; Shallow-water equations; Numerical analysis/modeling

1. Introduction

As computer architectures evolve, new algorithms are often needed to make the best use of the latest equipment. For example, in the 1990s, there was a major transition from vector supercomputers to distributed memory clusters, and internode communication became the slowest part of a simulation. In global atmospheric models, this spurred a transition from spectral methods (Kiehl et al. 1998), which require global communications for the transforms between physical and spectral space, to methods such as finite volume (Thuburn et al. 2009) and spectral element (Dennis et al. 2005), which only require local communication with nearby processors.

In recent years, there has been another major change in the landscape of supercomputer architectures, as the main driver for sales became machine learning (ML) and artificial intelligence (AI) applications. Tensor cores are a new generation of chips specifically designed for AI and deep learning, such as the NVIDIA Volta, Turing, and Ampere classes of graphics processing units (GPUs). Because AI, rather than computational physics, is driving the direction of commodity architecture, developers of numerical methods must seek out algorithms that are performant on this new hardware. The AI revolution has also spurred the development of libraries that are specifically tuned to run AI algorithms, such as neural networks, on tensor cores. Here, we present recently developed numerical methods that leverage tensor networks. These methods manipulate large-scale data based on generalizations of the singular value decomposition to higher dimensional tensor arrays.

A second driver of new algorithm development is to greatly increase the speed and resolution of global climate simulations. High-resolution global simulations are now typically 6–10 km in the ocean and atmosphere (Caldwell et al. 2019), and the latest cloud-resolving simulations are at 3-km resolution (Donahue et al. 2024). This involves millions of horizontal cells and up to 128 vertical layers. In addition, climate research requires long simulations for spinup and ensembles of simulations to investigate the intrinsic variability of the climate system (Kay et al. 2015). All these factors taken together produce the "curse of dimensionality," where simulation campaigns in climate science require many months on large supercomputers.

Tensor networks (TNs) (Cichocki 2014) are a promising new approach that mitigate the effects of the curse of dimensionality and may also take advantage of specialized AI hardware. TNs are a generalization of matrix factorizations to higher dimensions, whereby multidimensional data structures (tensors) are decomposed into manageable blocks. The computations normally performed on the large dataset (e.g., finite differences) can alternatively be performed on these smaller tensor components, drastically reducing computational costs. The most popular TN method is known as the tensor train (TT). In the TT format, the large dataset is decomposed into a sequence of lower-dimensional tensors (cores), linked together in a chain (train), to efficiently represent and manipulate large-scale data (Oseledets and Tyrtyshnikov 2010).

TN techniques show great promise to attack the curse of dimensionality across a large range of problems. Just in the last few years, tensor methods have been used to model the Navier–Stokes equations in a number of standard test cases:

Supplemental information related to this paper is available at the Journals Online website: https://doi.org/10.1175/MWR-D-24-0165.s1.

Corresponding author: M. Engin Danis, danis@lanl.gov

DOI: 10.1175/MWR-D-24-0165.1

© 2025 American Meteorological Society. This published article is licensed under the terms of the default AMS reuse license. For information regarding reuse of this content and general copyright information, consult the AMS Copyright Policy (www.ametsoc.org/PUBSReuseLicenses).

Brought to you by Los Alamos National Laboratory | Unauthenticated | Downloaded 10/23/26 05:39 PM UTC

a backward-facing step (Demir et al. 2024), a lid-driven cavity (Kiffner and Jaksch 2023), a turbulent flow in a channel (von Larcher and Klein 2019), and a three-dimensional Taylor–Green vortex (Gourianov 2022). In the recent work, we used TT methods to accelerate compressible flow simulations by up to 1000 times (Danis et al. 2025) and solved the time-independent Boltzmann neutron transport equation with tensor networks (Truong et al. 2024). These successes show that a tensor-based approach to fluids problems is both possible and promises greatly increased model efficiency. There are similarities between the methods presented here to those in Danis et al. (2025), but the present work serves to compliment (Danis et al. 2025) by extending these TN techniques and results to models of oceanic or atmospheric circulation, which has not been done to date. A discussion comparing methods from Danis et al. (2025) to those in the present work can be found in section 4b. Successful compression and speedup of geophysical fluid simulations via TN has the promise to radically alter the landscape of weather and climate modeling, allowing researchers to explore higher resolutions and larger ensembles.

Any new numerical method for atmosphere and ocean models must progress through a sequence of verification steps to be accepted by the community. The shallow-water equations (SWEs) are a reduced equation set used as a first step for modeling geophysical fluids at the climate scale (Thuburn et al. 2009; Weller et al. 2009; Archibald et al. 2011; Lilly et al. 2023). In particular, the SWEs serve as a suitable starting point for methods targeting layered Boussinesq models, which can be thought of as a vertical stack of shallow-water models with additional vertical and surface processes. The SWEs contain the relevant dynamics of atmospheric and oceanic flows: the Coriolis force and pressure gradient term for geostrophic balance, as well as horizontal advection of momentum and mass. At the same time, the SWEs are simple enough for rapid code development. Critically, the SWEs may be tested against exact solutions during model development (Bishnu et al. 2024), which is not the case for the subsequent levels of complexity in the development sequence (Petersen et al. 2015). The key assumptions of the SWEs are that horizontal scales of motion are much larger than the vertical, which means it is hydrostatic, and that the fluid is incompressible so that it has uniform density (Cushman-Roisin and Beckers 2011). This conveniently avoids the complexities of the equation of state, moist dynamics and clouds in the atmosphere, and vertical advection. Published test sets using the SWEs have become an essential component of model development and verification (Williamson et al. 1992; Calandrini et al. 2021).

This article assesses the computational advantages of TN in modeling the SWEs across a range of test cases. In particular, we focus on TT methods for high-order finite-volume methods for the SWEs. The paper is organized as follows. In section 2, we review the SWEs and our high-order methods for solving them. In section 3, we cover the basics of the TT decomposition, and in section 4, we discuss how the finite-volume scheme can be formulated in the TT format. We end with section 5 covering the results for a series of test cases.

2. Governing equations and the numerical method

In this section, we will review the SWEs and the finite-volume method used to solve these equations. This discussion will only involve essential information for a typical finite-volume implementation on traditional grids to lay the ground for the tensor-train implementation.

a. Shallow-water equations

In this study, we consider both linear and nonlinear SWEs with a flat bottom topography. Both equations are solved in the conservative form:

$$\frac{\partial \mathbf{U}}{\partial t} + \frac{\partial \mathbf{F}}{\partial x} + \frac{\partial \mathbf{G}}{\partial y} = \mathbf{S}, \tag{1}$$

where $\mathbf{U}$ is the vector of conserved variables, $\mathbf{F}$ and $\mathbf{G}$ are the fluxes in x and y directions, and $\mathbf{S}$ is the source term.

In the linear case, Eq. (1) is solved with

$$\mathbf{U} = \begin{pmatrix} \eta \\ u \\ v \end{pmatrix}, \quad \mathbf{F} = \begin{pmatrix} Hu \\ g\eta \\ 0 \end{pmatrix}, \quad \mathbf{G} = \begin{pmatrix} Hv \\ 0 \\ g\eta \end{pmatrix}, \quad \mathbf{S} = \begin{pmatrix} 0 \\ fv \\ -fu \end{pmatrix}, \tag{2}$$

where the vector of conserved variables $\mathbf{U}$ consists of the surface elevation η, the x velocity u, and the y velocity v. Furthermore, H is the mean depth of the fluid at rest, f is the Coriolis parameter, and g is the acceleration of gravity.

In the nonlinear case, Eq. (1) is solved with

$$\mathbf{U} = \begin{pmatrix} h \\ hu \\ hv \end{pmatrix}, \quad \mathbf{F} = \begin{pmatrix} hu \\ hu^2 + \frac{1}{2}gh^2 \\ huv \end{pmatrix}, \quad \mathbf{G} = \begin{pmatrix} hv \\ huv \\ hv^2 + \frac{1}{2}gh^2 \end{pmatrix}, \quad \mathbf{S} = \begin{pmatrix} 0 \\ fhv \\ -fhu \end{pmatrix}, \tag{3}$$

where h is the fluid layer thickness.

b. High-order finite-volume method for hyperbolic conservation laws

For simplicity, let us consider the two-dimensional scalar hyperbolic conservation law:

$$\frac{\partial u}{\partial t} + \frac{\partial f(u)}{\partial x} + \frac{\partial g(u)}{\partial y} = 0, \tag{4}$$

where u is a generic conserved variable (not to be confused with the x velocity) and $f(u)$ and $g(u)$ are the fluxes in x and y directions, respectively, and we assume that proper initial and boundary conditions are provided. On a uniform mesh with grid spacing Δx and Δy in x and y directions, a finite-volume method solves Eq. (4) for the cell averages of u in a given cell (i, j):

$$\tilde{\bar{u}}_{i,j} = \frac{1}{\Delta x \Delta y} \int_{x_{i-1/2}}^{x_{i+1/2}} \int_{y_{j-1/2}}^{y_{j+1/2}} u dx dy, \tag{5}$$

where $\bar{u}$ denotes the cell average in x and $\tilde{u}$ denotes the cell average in y.

Brought to you by Los Alamos National Laboratory | Unauthenticated | Downloaded 10/23/26 05:39 PM UTC

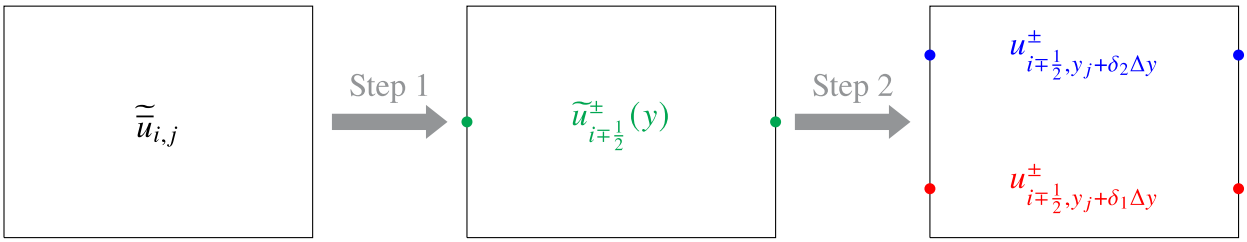


FIG. 1. Step-by-step third-order reconstruction from cell averages in the x direction.

In this spirit, the semidiscrete cell-averaged form of Eq. (4) for a cell (i, j) is given as

$$\frac{d\tilde{\bar{u}}_{i,j}}{dt} + \frac{1}{\Delta x\Delta y}\int_{y_{j-1/2}}^{y_{j+1/2}}\{f[u(x_{i+1/2}, y)] - f[u(x_{i-1/2}, y)]\}dy + \frac{1}{\Delta x\Delta y}\int_{x_{i-1/2}}^{x_{i+1/2}}\{g[u(x, y_{j+1/2})] - g[u(x, y_{j-1/2})]\}dx = 0, \quad (6)$$

which can be approximated as

$$\frac{d\tilde{\bar{u}}_{i,j}}{dt} + \frac{\hat{f}_{i+1/2,j} - \hat{f}_{i-1/2,j}}{\Delta x} + \frac{\hat{g}_{i,j+1/2} - \hat{g}_{i,j-1/2}}{\Delta y} = 0, \quad (7)$$

where the numerical fluxes $\hat{f}_{i\pm1/2,j}$ and $\hat{g}_{i,j\pm1/2}$ approximate the surface integrals by the one-dimensional Gauss–Legendre quadrature rule with quadrature points δ_m and weights w_m:

$$\hat{f}_{i\pm1/2,j} = \sum_m w_m \hat{f}(u^-_{i\pm1/2,y_j+\delta_m\Delta y}, u^+_{i\pm1/2,y_j+\delta_m\Delta y}),$$
$$\hat{g}_{i,j\pm1/2} = \sum_m w_m \hat{g}(u^-_{x_i+\delta_m\Delta x,j\pm1/2}, u^+_{x_i+\delta_m\Delta x,j\pm1/2}), \quad (8)$$

where $\hat{f}(u^-, u^+)$ and $\hat{g}(u^-, u^+)$ denote the local Lax–Friedrichs flux. In the x direction, for example, the local Lax–Friedrichs flux is defined as

$$\hat{f}(u^-, u^+) = \frac{1}{2}[f(u^-) + f(u^+)] - \frac{\lambda}{2}(u^+ - u^-), \quad (9)$$

where $u^\pm$ are the pointwise values at the cell interfaces approximated by a high-order reconstruction method from the cell averages $\tilde{\bar{u}}_{i,j}$ and $\lambda = \max_{u\in(u^-,u^+)}|f'(u)|$.

c. High-order reconstructions

In this paper, we implement the third-order upwind-biased (Upwind3), fifth-order upwind-biased (Upwind5), and fifth-order weighted essentially nonoscillatory (WENO5) reconstruction methods. Upwind methods are based on solution reconstruction using the ideal weights obtained from the best polynomial approximation that matches the cell averages in the relevant computational stencil. However, they become extremely oscillatory when the numerical solution develops discontinuities, such as shock waves, which makes the numerical solution eventually unstable. For those cases, WENO methods provide a robust numerical solution near discontinuities while maintaining the high-order accuracy in the smooth regions of the solution by blending the ideal reconstruction weights with smoothness indicators to obtain a nonlinear reconstruction. We refer the interested readers to Shu (1998) for more details.

To obtain a high-order two-dimensional finite-volume discretization, we follow the dimension-by-dimension reconstruction method of Shu (1998) and Shi et al. (2002). This involves two one-dimensional reconstruction steps for each direction. In Fig. 1, the reconstruction procedure for the third-order reconstruction in the x direction is depicted, for which we only use two Gauss quadrature points for approximating the surface integrals. Let $\bar{}$ and $\tilde{}$ denote the cell averages in the x and y directions, respectively. Then, starting with the cell average $\tilde{\bar{u}}_{i,j}$, step 1 is to perform the first one-dimensional reconstruction in the x direction at $x = x_{i\pm1/2}$; this gives one-dimensional cell averages $\tilde{u}^{\pm}_{i\mp1/2}$. Then, in step 2, we perform a second one-dimensional reconstruction, now in the y direction, to obtain pointwise values $u^{\pm}_{i\mp1/2,y_j+\delta_2\Delta y}$ at the Gauss quadrature points at $x = x_{i\pm1/2}$ and $y \in \{y_j + \delta_1\Delta y, y_j + \delta_2\Delta y\}$.

The fifth-order reconstruction steps 1 and 2 for Upwind5 and WENO5 are similar; we present the details for all three reconstructions in appendixes A and B.

d. Finite-volume method for the shallow-water equations

In this study, we solve the cell-averaged SWEs,

$$\frac{\partial\tilde{\bar{\mathbf{U}}}_{i,j}}{\partial t} + \frac{\hat{\mathbf{F}}_{i+1/2,j} - \hat{\mathbf{F}}_{i-1/2,j}}{\Delta x} + \frac{\hat{\mathbf{G}}_{i,j+1/2} - \hat{\mathbf{G}}_{i,j-1/2}}{\Delta y} = \tilde{\bar{\mathbf{S}}}_{i,j}, \quad (10)$$

on a uniform Cartesian mesh using the finite-volume method where the fluxes vectors are computed by the Gauss–Legendre quadrature rule:

$$\hat{\mathbf{F}}_{i\pm1/2,j} = \sum_m w_m \hat{\mathbf{F}}(\mathbf{U}^-_{i\pm1/2,y_j+\delta_m\Delta y}, \mathbf{U}^+_{i\pm1/2,y_j+\delta_m\Delta y}),$$
$$\hat{\mathbf{G}}_{i,j\pm1/2} = \sum_m w_m \hat{\mathbf{G}}(\mathbf{U}^-_{x_i+\delta_m\Delta x,j\pm1/2}, \mathbf{U}^+_{x_i+\delta_m\Delta x,j\pm1/2}), \quad (11)$$

with the local Lax–Friedrichs flux

$$\hat{\mathbf{F}}(\mathbf{U}^-, \mathbf{U}^+) = \frac{1}{2}[\mathbf{F}(\mathbf{U}^-) + \mathbf{F}(\mathbf{U}^+)] - \frac{\lambda_F}{2}(\mathbf{U}^+ - \mathbf{U}^-),$$
$$\hat{\mathbf{G}}(\mathbf{U}^-, \mathbf{U}^+) = \frac{1}{2}[\mathbf{G}(\mathbf{U}^-) + \mathbf{G}(\mathbf{U}^+)] - \frac{\lambda_G}{2}(\mathbf{U}^+ - \mathbf{U}^-), \quad (12)$$

where λ_F and λ_G are the maximum eigenvalues of the flux Jacobians $|\partial\mathbf{F}/\partial\mathbf{U}|$ and $|\partial\mathbf{G}/\partial\mathbf{U}|$, respectively. The high-order

Brought to you by Los Alamos National Laboratory | Unauthenticated | Downloaded 10/23/26 05:39 PM UTC

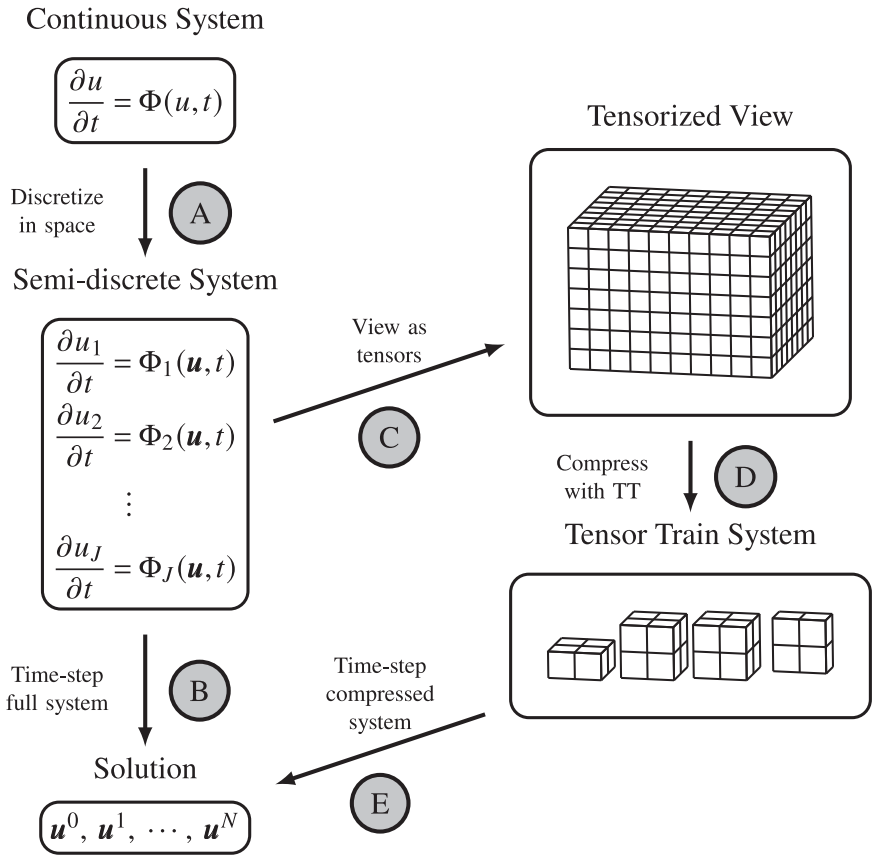


FIG. 2. TT methods find speedups by avoiding time stepping the full discrete system directly. Standard methods involve (a) discretizing the PDE in space, followed by (b) the use of an iterative time-stepping algorithm. TT methods can be understood to instead (c) treat the discrete system as a large-dimensional tensor, on which (d) a TT is fit. Speedup is found by (e) time stepping in the reduced-dimensional latent space of the TT, which scales linearly with the size of the problem instead of polynomially as in (b).

reconstruction is performed by applying the procedures discussed in section 2c to $\mathbf{U}$ in a component-by-component fashion.

3. Tensor-train decomposition

In this section, we briefly introduce the tensor notation and the tensor-train manipulation techniques we apply in this work. The goal of the machinery introduced in this section and the methods described in section 4 is to use the tensor-train format to obtain an approximation to the model state that is significantly compressed and then to evolve that compressed state forward in time. By performing all necessary operations in this compressed space, we can greatly reduce the number of floating point operations required to advance the model, thereby obtaining significant computational speedup. Figure 2 serves to provide a high-level explanation of the goal of tensor-train methods like the ones used here.

For clarity, in the context of this work, a tensor is exactly a multidimensional array of arbitrary dimension, consisting of real entries, subject to nonlinear pointwise operations and linear transformations. For a positive integer d and a sequence of mode sizes $n_1, n_2, \ldots, n_d$, a tensor $\mathcal{X}$ of the given shape is an element of $\mathbb{R}^{n_1 \times n_2 \times \cdots \times n_d}$. In this way, a $n_1 \times n_2$ matrix is a two-dimensional tensor in $\mathbb{R}^{n_1 \times n_2}$, and a length n_1 vector is a one-dimensional tensor in $\mathbb{R}^{n_1}$. A d-dimensional tensor in $\mathbb{R}^{n_1 \times n_2 \times \cdots \times n_d}$ is indexed by a multiindex $\mathbf{i} = (i_1, i_2, \ldots i_d)$, where $i_k \in \{1, 2, \ldots, n_k\}$ for all $k = 1, 2, \ldots, d$; this is written as $\mathcal{X}(i_1, i_2, \ldots, i_d) \in \mathbb{R}$.

a. Tensor train

Tensor train, or the TT format of a tensor, represents a tensor of arbitrary dimension as a product of so-called cores, which are either two- or three-dimensional tensors (Oseledets and Tyrtyshnikov 2010). Generally, a TT representation $\mathcal{X}_{\mathrm{TT}}$ of a d-dimensional tensor $\mathcal{X}$ is defined as

$$\mathcal{X}_{\mathrm{TT}}(i_1, \ldots, i_d) = \sum_{\alpha_1, \ldots, \alpha_{d-1}}^{r_1, \ldots, r_{d-1}} \mathcal{G}_1(1, i_1, \alpha_1)\mathcal{G}_2(\alpha_1, i_2, \alpha_2)\ldots\mathcal{G}_{d-1} \times (\alpha_{d-2}, i_{d-1}, \alpha_{d-1})\mathcal{G}_d(\alpha_{d-1}, i_d, 1), \quad (13)$$

Brought to you by Los Alamos National Laboratory | Unauthenticated | Downloaded 10/23/26 05:39 PM UTC

where $\|\mathcal{X}_{\mathrm{TT}} - \mathcal{X}\|_F < \varepsilon$, for a prescribed value of ε. Here, $\|\cdot\|_F$ is the Frobenius norm. The entries of the integer array $\mathbf{r} = [r_1, \ldots, r_{d-1}]$ are called TT ranks, and $\mathcal{G}_k$ are called TT cores. Equivalently, we can also denote the TT format by the multiple matrix product:

$$\mathcal{X}_{\mathrm{TT}}(i_1, i_2, \ldots, i_d) = \mathbf{G}_1(i_1)\mathbf{G}_2(i_2)\ldots\mathbf{G}_d(i_d), \tag{14}$$

where each term, $[\mathbf{G}_k(i_k)]\alpha_{k-1}, \alpha_k$, $i_k = 1, 2, \ldots, n_k$, $k = 1, 2, \ldots, d$, is a matrix of size $r_{k-1} \times r_k$ (where $r_0 = r_d = 1$). Therefore, the $\mathcal{G}_k(:, i_k, :)$ are a set of matrix slices $\mathbf{G}_k(i_k)$ that are labeled with the single index i_k. Since each TT core only depends on a single-mode index of the full tensor $\mathcal{X}$, e.g., i_k, the TT format effectively embodies a discrete separation of variables.

Assuming that $n_k = \mathcal{O}(n)$ and $r_k = \mathcal{O}(r)$ for some nonnegative integers n and r, and for all $k = 1, 2, \ldots, d$, the total number of elements that the TT format stores is proportional to $\mathcal{O}[2nr + (d-2)nr^2]$, which is linear with the number of dimensions d. In this way, the TT ranks $\mathbf{r} = [r_1, \ldots, r_{d-1}]$ quantify the effectiveness of the TT compression. When the TT ranks are relatively small with respect to the problem size, a TT-based approach is referred to as a low-rank approximation (Bachmayr 2023).

In the case that $\mathcal{X}$ is a matrix, its TT format is simplified to the following:

$$\mathcal{X}_{\mathrm{TT}}(i_1, i_2) = \mathbf{G}_1(i_1)\mathbf{G}_2(i_2). \tag{15}$$

Given that the shallow-water problems we investigate in this work have two spatial dimensions, this decomposition will be the one we use to represent the solutions at each time step.

Note that both a tensor $\mathcal{X}$ and its TT representation $\mathcal{X}_{\mathrm{TT}}$ share the same set of indices; if $\mathcal{X}$ represents data at a given set of grid points, $\mathcal{X}_{\mathrm{TT}}$ also represents data at the same set of grid points. The difference is that the TT representation does not necessarily explicitly store data at each grid point but rather returns these data via the tensor contraction operation given in Eq. (13). In this way, both $\mathcal{X}$ and $\mathcal{X}_{\mathrm{TT}}$ share the same shape but do not explicitly store the same number of elements.

Tensor train is an advantageous low-rank format for many reasons, but perhaps one of the most important is its compatibility with basic linear operations. For example, arithmetic operations such as addition, scalar multiplication, and pointwise multiplication can be performed on the TT cores without the need to form the complete tensor (Oseledets 2011). Importantly, tensor trains are also compatible with linear transformations via linear operators. That is, given a tensor $\mathcal{X}$ and a linear operator L such that $L(\mathcal{X}) = \mathcal{Y}$, one can form a matrix product operator (MPO) in the TT format of L, call this A_L, such that $A_L\mathcal{X}_{\mathrm{TT}} = \mathcal{Y}_{\mathrm{TT}}$. Here, the operation $A_L\mathcal{X}_{\mathrm{TT}}$ is a generalization of the matrix product that is computed entirely in the TT format (Oseledets 2011).

However, when these basic operations are performed in the TT format, the rank of the resulting TT will grow relative to that of the input TTs. To reap the benefit of low-rank compression for iterative schemes, a method known as TT rounding has been developed. We discuss this next.

b. TT rounding

Given a tensor $\mathcal{X}$ represented in the TT format $\mathcal{X}_{\mathrm{TT}}$ with TT ranks $\mathbf{r} = [r_1, \ldots, r_{d-1}]$, one often wishes to find an even more compact TT representation $\mathcal{Y}_{\mathrm{TT}}$, with TT ranks $\mathbf{r}' = [r'_1, \ldots, r'_{d-1}]$ such that $r'_i \leq r_i$ for $i = 1, \ldots, d-1$. This is because, as discussed above, as certain common operations are applied to TTs, the ranks of resultant TTs can quickly grow, causing the benefits of the initial compression to disappear. To obtain $\mathcal{Y}_{\mathrm{TT}}$, such that $\|\mathcal{X}_{\mathrm{TT}} - \mathcal{Y}_{\mathrm{TT}}\|_F < \varepsilon_{\mathrm{TT}}$ for a prescribed $\varepsilon_{\mathrm{TT}}$, one can apply a so-called TT-rounding procedure. This type of procedure is also called truncation or recompression and is discussed in detail in Oseledets (2011). To illustrate the procedure in a simple case, consider the TT of a matrix like that in Eq. (15). In this case, the TT-rounding algorithm consists of two steps. First, the second core $\mathbf{G}_2$ is orthogonalized using RQ decomposition.[1] Thereafter, singular value decomposition (SVD) truncation at tolerance $\varepsilon_{\mathrm{TT}}$ is applied to the product $\mathbf{G}_1\mathbf{R}$ to arrive at a new decomposition $\mathcal{Y}_{\mathrm{TT}}$. In this study, we denote TT rounding by $\mathcal{Y}_{\mathrm{TT}} = \mathrm{round}(\mathcal{X}_{\mathrm{TT}}, \varepsilon_{\mathrm{TT}})$. The computational cost of TT rounding is nontrivial, especially for TTs with large TT ranks. As such, we choose carefully when to apply TT rounding, as described in section 4d.

c. TT cross interpolation

It should be noted that not all operations can be directly computed in the TT format. Relevant examples discussed in section 4 include $\sqrt{\mathcal{X}_{\mathrm{TT}}}$, $|\mathcal{X}_{\mathrm{TT}}|$, and $1/\mathcal{X}_{\mathrm{TT}}$. However, nonlinear quantities such as these can be efficiently approximated via so-called TT cross interpolation. TT cross interpolation is a technique used to construct a TT representation of a tensor without needing to form the entire tensor explicitly. This method is particularly valuable when dealing with very large tensors or in situations where calculations are impractical due to limitations in TT arithmetic. Stemming from the skeleton (or CUR) decomposition (Mahoney and Drineas 2009), in combination with the maximum volume principle (Goreinov et al. 2010), heuristic cross-interpolation algorithms for tensor train, such as alternating minimal energy (AMEn) (Dolgov and Savostyanov 2014) or density matrix renormalization group (DMRG) (Savostyanov and Oseledets 2011), have been developed. In this work, we use an implementation of AMEn algorithm, amen_cross, which is available in MATLAB TT toolbox (Oseledets 2014).

4. Tensorization of the finite-volume scheme

In this section, we will discuss the tensorization of the finite-volume method for the shallow-water equations.

a. TT finite-volume (TT-FV) methods for hyperbolic conservation laws

We follow the methods discussed in Danis et al. (2025). Start with the full-tensor form of the SWEs with "loop indices":

[1] RQ decomposition is analogous to QR decomposition, except that the rows of the input matrix are orthogonalized, rather than the columns.

Brought to you by Los Alamos National Laboratory | Unauthenticated | Downloaded 10/23/26 05:39 PM UTC

$$\frac{\partial \tilde{\tilde{\mathbf{U}}}_{i,j}}{\partial t} + \frac{\hat{\mathbf{F}}_{i+1/2,j} - \hat{\mathbf{F}}_{i-1/2,j}}{\Delta x} + \frac{\hat{\mathbf{G}}_{i,j+1/2} - \hat{\mathbf{G}}_{i,j-1/2}}{\Delta y} = \tilde{\tilde{\mathbf{S}}}_{i,j}. \tag{16}$$

On a structured Cartesian mesh, we can introduce the shift operators in x and y and rewrite the flux terms as

$$\begin{aligned} \hat{\mathbf{F}}_{i+(1/2),j} &= T^x_{i,j}\hat{\mathbf{F}}_{i-(1/2),j}, \\ \hat{\mathbf{G}}_{i,j+(1/2)} &= T^y_{i,j}\hat{\mathbf{G}}_{i,j-(1/2)}. \end{aligned} \tag{17}$$

Substituting Eq. (17) into Eq. (16) and dropping the loop indices, we obtain the vectorized form of the SWEs, which we will refer to as the "full-tensor" form:

$$\frac{\partial \tilde{\tilde{\mathbf{U}}}}{\partial t} + \frac{1}{\Delta x}(T^x - 1)\hat{\mathbf{F}} + \frac{1}{\Delta y}(T^y - 1)\hat{\mathbf{G}} = \tilde{\tilde{\mathbf{S}}}. \tag{18}$$

Note that terms in Eq. (18) correspond to two-dimensional prestored arrays. Generally speaking, the full-tensor approach is slower than using the loop indices unless explicitly parallelized or vectorized. However, the full-tensor format naturally lends itself to compression of the total degrees of freedom through the TT format. Specifically, we can simply replace the full-tensor terms with their TT counterparts:

$$\frac{\partial \tilde{\tilde{\mathbf{U}}}_{\mathrm{TT}}}{\partial t} + \frac{1}{\Delta x}(T^x - 1)\hat{\mathbf{F}}_{\mathrm{TT}} + \frac{1}{\Delta y}(T^y - 1)\hat{\mathbf{G}}_{\mathrm{TT}} = \tilde{\tilde{\mathbf{S}}}_{\mathrm{TT}}. \tag{19}$$

In Eq. (19), all the terms are now in the TT format. This includes the conserved variables, the numerical fluxes, and the shift operators. The arithmetic operations such as the MPO multiplication $(T^x - 1)\hat{\mathbf{F}}_{\mathrm{TT}}$ and addition are all carried out in the TT format as discussed in section 3. To simplify notation, we continue to write the shift operators, now MPOs in the TT format, as T^x and T^y.

To perform the TT evolution of the SWEs, the tensor-train terms of Eq. (19) as well as the weights of the chosen finite-volume method must be computed in an efficient manner. The numerical fluxes $\hat{\mathbf{F}}_{\mathrm{TT}}$ and $\hat{\mathbf{G}}_{\mathrm{TT}}$ as well as the (WENO) finite-volume weights require careful consideration; these terms involve nonlinear operations on these TTs. We discuss this in the next subsections.

b. *Computing the fluxes in the tensor-train format*

To continue the formulation of our TT finite-volume methods, we need to specify how the numerical fluxes are efficiently calculated in the TT format.

In this study, we implement the TT format of the local Lax–Friedrichs flux similar to that suggested by Danis and Alexandrov (2023). For example, the fluxes in the x direction will be computed in the TT format as

$$\hat{\mathbf{F}}_{\mathrm{TT}}(\mathbf{U}^-_{\mathrm{TT}}, \mathbf{U}^+_{\mathrm{TT}}) = \frac{1}{2}[\mathbf{F}(\mathbf{U}^-_{\mathrm{TT}}) + \mathbf{F}(\mathbf{U}^+_{\mathrm{TT}})] - \frac{\lambda_{F,\mathrm{TT}}}{2}(\mathbf{U}^+_{\mathrm{TT}} - \mathbf{U}^-_{\mathrm{TT}}). \tag{20}$$

However, the linear and nonlinear SWEs differ in the computation of each individual term in Eq. (20). For the linear SWEs, the physical flux terms, $\mathbf{F}(\mathbf{U}^{\pm}_{\mathrm{TT}})$, can be directly computed, without relying on special considerations such as TT cross interpolation. Additionally, the eigenvalue $\lambda_{F,\mathrm{TT}} = \sqrt{gH}$ is a constant for the linear equations. In contrast, for the nonlinear SWE equations, $1/h_{\mathrm{TT}}$ must be computed for the physical fluxes $\mathbf{F}$ and $\mathbf{G}$ (to recover u from the conserved quantity hu), and $\lambda_{F,\mathrm{TT}} = |u_{\mathrm{TT}}| + \sqrt{gh_{\mathrm{TT}}}$ and $\lambda_{G,\mathrm{TT}} = |v_{\mathrm{TT}}| + \sqrt{gh_{\mathrm{TT}}}$ must be computed as the eigenvalues of the flux Jacobians of $\mathbf{F}$ and $\mathbf{G}$, respectively. These nonlinear functions of the TTs cannot be computed directly; therefore, we employ the AMEn method to compute eigenvalues $\lambda_{F,\mathrm{TT}}$ and $\lambda_{G,\mathrm{TT}}$ and the Taylor series approximation, given in alg:taylor-inverse, to compute the reciprocal of the tensor train h_{TT}.

Algorithm 1: Taylor series approximation to tensor-train reciprocal.

Data: x_{TT}, $\varepsilon_{TT} > 0$
Result: $y_{TT} = 1/x_{TT}$
$y_{TT} \leftarrow 1$;
$\Delta y_{TT} \leftarrow 1$;
$Err \leftarrow 1$;
$N \leftarrow \mathrm{numel}(x_{TT})$;
$x_{avg} \leftarrow \mathrm{sum}(x_{TT})/N$;
$\tilde{x}_{TT} \leftarrow \mathrm{round}(\mathbf{1} - x_{TT}/x_{avg}, \varepsilon_{TT})$;
while $Err > \varepsilon_{TT}$ **do**
    $\Delta y_{TT} \leftarrow \mathrm{round}(\Delta y_{TT} * \tilde{x}_{TT}, \varepsilon_{TT})$;
    $y_{TT} \leftarrow \mathrm{round}(y_{TT} + \Delta y_{TT}, \varepsilon_{TT})$;
    $Err \leftarrow \|\Delta y_{TT}\|_F/\sqrt{N}$;
$y_{TT} \leftarrow y_{TT}/x_{avg}$;

After obtaining $1/h_{\mathrm{TT}}$, the components of the flux vectors in Eq. (3) can be directly computed. Note that we could have also employed a TT cross-interpolation method to compute $1/h_{\mathrm{TT}}$ instead of algorithm 1. In the numerical examples considered in this study, however, we found that Taylor series approximation to $1/h_{\mathrm{TT}}$ is as fast as the AMEn method. This is possibly because the shallow water layer thickness h can be decomposed as a superposition of a large mean value and small amplitude oscillations, i.e., $h = H + \eta(x, y, t)$, where $|\eta| \ll H$, which leads to a very fast and robust convergence of the Taylor series approximation for $1/h_{\mathrm{TT}}$.

Note also that this method is different than the LF-cross method developed in Danis et al. (2025), where the complete flux vector of the compressible Euler equations is computed with a single cross interpolation using the AMEn method. In the context of the finite-difference method for solving compressible flow equations, the LF-cross method was reported to be faster than a similar method presented here that approximates $1/\rho_{\mathrm{TT}}$ (reciprocal of density) with the TT cross interpolation to compute the flux vector components of compressible Euler equations directly. This was thought to be due to slow TT rounding while each flux vector component was estimated. However, for the finite-volume method for solving the shallow-water equations, we found that the LF-cross approach is considerably slower than the present approach. This might be due to two reasons. First, the compressible Euler equations have more conserved variables than the SWEs, and therefore, more floating point operations are needed to compute fluxes in compressible flow, meaning that the rounding routine will be called more often, and each call will be more expensive due to the

Brought to you by Los Alamos National Laboratory | Unauthenticated | Downloaded 10/23/26 05:39 PM UTC

additional cost of quadrature rules used in high-order finite-volume schemes. Second, computing $1/h_{\mathrm{TT}}$ for the SWEs is likely a much simpler task than computing $1/\rho_{\mathrm{TT}}$ for the compressible flow. As mentioned above, in the case of the SWEs, h is simply a superposition of a mean depth and small-amplitude waves, but in the compressible flow, the density field ρ can have large variations and even, discontinuities such as strong shock waves. Therefore, the nonlinear flux calculations should be designed or selected according to the particular application, especially when the computation of the reciprocal of a tensor train is required. Looking forward to extending these methods to layered Boussinesq models appropriate for ocean and atmosphere circulation at the climate scale, we expect that the method of computing $1/h_{\mathrm{TT}}$ presented here in algorithm 1 will extend naturally; the barotropic mode of a layered Boussinesq model is exactly the SWEs. The baroclinic mode is given by a vertical stack of shallow-water models that communicate via vertical fluxes; it is likely that these will be well treated by the approach from algorithm 1 as well.

c. *High-order reconstructions in the tensor-train format*

Finally, to complete our TT finite-volume methods, we specify how the high-order reconstructions are handled in the TT format. Here, we consider two types of high-order variable reconstruction methods—linear (Upwind3 and Upwind5) and nonlinear (WENO5) reconstructions as discussed in section 2c and appendixes A and B. For linear reconstruction schemes, we are able to directly manipulate the TT cores, which is more computationally efficient than manipulating the entire TT. However, for nonlinear WENO reconstruction, this is not possible because of the presence of inverse quadratic terms in in the WENO weights. Therefore, we employ TT cross interpolation via the AMEn method for the WENO reconstruction.

1) LINEAR RECONSTRUCTION METHODS

The linear reconstruction in the TT format in a given direction is applied only to the TT core corresponding to that direction, which significantly reduces the computational costs. To exploit this, we modify the algorithm presented in appendixes A and B. At a high level, the algorithm consists of two steps: step 1 is "decell" averaging over x followed by step 2, decell averaging over y. Note that, in the full-tensor format, step 1 is followed by step 2 for each time a reconstruction is needed along a cell interface. This means that, in a two-dimensional setting, step 2 is applied twice to close the cell boundaries. However, this is not needed in the TT format.

To illustrate the idea, we adopt the notation used in Oseledets (2011)and denote the elementwise values of a cell-averaged tensor $\tilde{\bar{u}}$ as

$$\tilde{\bar{u}}(i, j) = \bar{u}_1(i)\tilde{u}_2(j). \tag{21}$$

Note that the first core $\bar{u}_1$ is written with a bar to denote the cell-averaging operator in x and the second core $\tilde{u}_2$ is written with a tilde to denote the cell-averaging operator in y. This implies that step 2 in appendixes A and B can be applied to each core independently and even simultaneously.

TT reconstruction step 1 starts with reconstructing the cores at the quadrature points $u_1(x_i + \delta_m\Delta x)$ and $u_2(x_j + \delta_m\Delta y)$. This step is almost identical to step 2 in appendixes A and B, except for the fact that the reconstruction is only applied to TT cores rather than to the full tensor.

TT reconstruction step 2 is similar to step 1 in appendixes A and B, and again, we only apply reconstruction to the TT cores. For example, a reconstruction in the x direction first computes $u_1(i \pm 1/2)^{\mp}$, and then, the result is combined with $u_2(x_j + \delta_m\Delta y)$ prepared in TT reconstruction step 1:

$$u(i \pm 1/2,\, y_j + \delta_m\Delta y)^{\mp} = u_1(i \pm 1/2)^{\mp} u_2(y_j + \delta_m\Delta y). \tag{22}$$

Similarly, for the reconstruction in the y direction, TT reconstruction step 2 computes $u_2(j \pm 1/2)^{\mp}$ and combines this with $u_1(x_i + \delta_m\Delta x)$ prepared in TT reconstruction step 1:

$$u(x_i + \delta_m\Delta x,\, j \pm 1/2)^{\mp} = u_1(x_i + \delta_m\Delta x) u_2(j \pm 1/2)^{\mp}. \tag{23}$$

2) NONLINEAR RECONSTRUCTION METHOD

The WENO scheme performs the nonlinear reconstruction using the TT cross interpolation by applying step 1 and step 2 of appendixes A and B in the same order. In fact, TT-WENO step 1 is similar to the finite-difference WENO cross method proposed in Danis et al. (2025), i.e., step 1 is applied in an equation-by-equation fashion using the TT cross interpolation as detailed in algorithm 2. Here, funWENO is a function that takes as input the x-direction 5-stencil $\mathbf{S} = \{(T^x)^{-2}\tilde{\bar{v}}, (T^x)^{-1}\tilde{\bar{v}}, \tilde{\bar{v}}, T^x\tilde{\bar{v}}, (T^x)^2\tilde{\bar{v}}\}$ of a conserved, cell-averaged quantity $\tilde{\bar{v}}$ and returns the cell-averaged values $\tilde{v}^{\pm}$ in the y direction. In plain language, funWENO applies step 1 of appendixes A and B for WENO5 to the full-tensor problem. The AMEn method is then able to approximate the application of this function to the TT quantity $\tilde{\bar{v}}_{\mathrm{TT}}$ without ever forming the full tensor.

Algorithm 2: TT-WENO Step 1 for the Decell averaging over x.

Data: A component of the cell-averaged conserved variable vector $\tilde{\bar{v}}_{TT}$, function *funWENO*, and the convergence criterion ε_{TT}.
Result: Cell-averaged $\tilde{v}^{\pm}_{TT}$ in y-direction at the interface locations.
1 Collect the 5-cell stencil of $\tilde{\bar{v}}_{TT}$ in the x-direction into the array. $\mathbf{S} = \{(T^x)^{-2}\tilde{\bar{v}}_{TT}, (T^x)^{-1}\tilde{\bar{v}}_{TT}, \tilde{\bar{v}}_{TT}, T^x\tilde{\bar{v}}_{TT}, (T^x)^2\tilde{\bar{v}}_{TT}\}$.
2 Set the initial guess $v_0 = \bar{v}_{TT}$.
3 Perform cross interpolation for + side: $\tilde{v}^{+}_{TT} = \mathrm{AMEn}(funWENO, \mathbf{S}, v_0, \varepsilon_{TT}, +1)$
4 Perform cross interpolation for − side: $\tilde{v}^{-}_{TT} = \mathrm{AMEn}(funWENO, \mathbf{S}, v_0, \varepsilon_{TT}, -1)$

Brought to you by Los Alamos National Laboratory | Unauthenticated | Downloaded 10/23/26 05:39 PM UTC

Similarly, TT-WENO step 2 applies step 2 of appendixes A and B for each conserved variable using the TT cross interpolation, see algorithm 3. However, it performs the WENO reconstruction at all quadrature points with a single cross interpolation. Similar to the above, *fun*WENO-quad applies step 2 of appendixes A and B for WENO5 to $\tilde{v}^{\pm}$.

Note that the AMEn cross routine in the MATLAB toolbox returns a TT of shape $1 \times N_x \times N_y \times N_q$, where N_x is the number of grid points in the x direction, N_y is the number of grid points in the y directions, and N_q is the number of quadrature points. Therefore, as a last step, it is reshaped and permuted to obtain a new TT of shape $1 \times N_x \times N_y N_q \times 1$ for the reconstruction in the x direction and $1 \times N_x \times N_q \times N_y \times 1$ for the reconstruction in the y direction.

Algorithm 3: TT-WENO Step 2 for the "de-cell" averaging in y.

Data: Cell-averaged $\tilde{v}^{\pm}_{TT}$ in y-direction obtained from TT-WENO Step 1, function *funWENOquad*, and the convergence criterion ε_{TT}.
Result: $v^{\pm}_{TT}$ at the interface locations.
1 Collect the 5-cell stencil of v_{TT} in the x-direction into the arrays $\mathbf{S}^{\pm} = \{(T^y)^{-2}\tilde{v}^{\pm}_{TT}, (T^y)^{-1}\tilde{v}^{\pm}_{TT}, \tilde{v}^{\pm}_{TT}, T_y\tilde{v}^{\pm}_{TT}, (T^y)^2\tilde{v}^{\pm}_{TT}\}$.
2 Set the initial guesses as $v^{\pm}_0 = \tilde{v}^{\pm}_{TT}$.
3 Perform cross interpolation for + side: $v^{+}_{TT} = \text{AMEn}(funWENOquad, \mathbf{S}^{+}, v^{+}_0, \varepsilon_{TT}, +1)$
4 Perform cross interpolation for − side: $v^{-}_{TT} = \text{AMEn}(funWENOquad, \mathbf{S}^{-}, v^{-}_0, \varepsilon_{TT}, -1)$
5 Reshape and permute $v^{\pm}_{TT}$to return two TTs of size $1 \times N_x \times N_y N_q \times 1$.

d. *Choosing a suitable* ε_{TT}

The choice of ε_{TT} determines the overall accuracy and the speed of the TT solver, as demonstrated in Danis et al. (2025). In this study, we modify the ε_{TT} formula slightly to accommodate both the third- and fifth-order schemes as

$$\varepsilon_{\text{TT}} = C_{\varepsilon}\frac{V^{1/2}\Delta x^{p-1/2}}{\max_{q\in \mathbf{U}_{\text{TT}}}\|q\|_F}, \tag{24}$$

where $\|\cdot\|_F$ is the Frobenius norm, p denotes the order of accuracy of the TT-FV scheme, V is the volume of the computational domain, and C_{ε} is a problem-dependent variable (see Danis et al. 2025).

Ideally, the choice of C_{ε} should be made to ensure that the TT truncation errors are less than or equal to the discretization errors of the underlying numerical scheme such that the formal convergence rate is maintained in the TT format. However, this requires an a priori knowledge of the discretization error, which is generally not available. If a very large C_{ε} is set, then the numerical TT solution will attain a relatively low-rank structure, but the formal accuracy of the underlying scheme will be lost. On the other hand, a very small C_{ε} will maintain the accuracy, but the TT ranks will be unnecessarily large. In the present study, thanks to the nondimensionalization of the SWEs, $C_{\varepsilon} = 1$ is found to be sufficient for obtaining high-order results without increasing the TT ranks in the majority of the numerical examples. Despite this success, however, choosing a suitable value for C_{ε} still remains as an empirical process. To determine C_{ε}, we recommend a procedure analogous to a grid independence study—a common practice applied in computational fluid dynamics (CFD) for identifying the finest grid resolution at which further refinement does not significantly alter the numerical solution. To ensure C_{ε} independence, one could start with a relatively large value of C_{ε} and incrementally decrease it until the numerical solution becomes independent of C_{ε}.

Note that if one of the conserved variables is exactly zero, choosing ε_{TT} according to Eq. (24) may still result in rank growth. This is primarily because ε_{TT} becomes comparable to the numerical noise generated for that conserved variable, which is known to result in rank growth. A robust workaround is obtained by replacing the maximum operator in the denominator of Eq. (24) with the norm of the relevant conserved variable itself. For example, consider the conserved variable $u_{\text{TT}} \in \mathbf{U}_{\text{TT}}$ of the linear SWE Eq. (2), for which we calculate

$$\varepsilon^{u}_{\text{TT}} = \min\left(10^{-3}, C_{\varepsilon}\frac{V^{1/2}\Delta x^{p-1/2}}{\|u\|_F}\right), \tag{25}$$

at the beginning of a time step. Note that the new dynamic formula is clipped at 10^{-3} to avoid accuracy loss if $\|u\|_F \to 0$, where the upper-bound 10^{-3} is empirically deemed sufficient to limit the TT truncation error such that the relative error $\|u - u^*\|_F/\|u\|_F \le 10^{-3}$ is achieved after performing $u = \text{round}(u^*, \varepsilon_{\text{TT}})$.

Equipped with Eq. (25), the third-order strong stability-preserving Runge–Kutta method (Shu and Osher 1988; Gottlieb et al. 2001) for a given semidiscrete form of u_{TT},

$$\frac{du_{\text{TT}}}{dt} = L(u_{\text{TT}}), \tag{26}$$

is then defined in the TT format as

$$\begin{aligned} u^{(1)}_{\text{TT}} &= \mathcal{F}(u^{n}_{\text{TT}}),\\ u^{n+1}_{\text{TT}} &= \text{round}\left\{\frac{1}{3}u^{n}_{\text{TT}} + \frac{2}{3}\mathcal{F}[u^{(2)}_{\text{TT}}], \varepsilon^{u}_{\text{TT}}\right\},\\ u^{(2)}_{\text{TT}} &= \text{round}\left\{\frac{3}{4}u^{n}_{\text{TT}} + \frac{1}{4}\mathcal{F}[u^{(1)}_{\text{TT}}], \varepsilon^{u}_{\text{TT}}\right\}, \end{aligned} \tag{27}$$

Brought to you by Los Alamos National Laboratory | Unauthenticated | Downloaded 10/23/26 05:39 PM UTC

TABLE 1. Summary of test cases.

Test case	Upwind3		Upwind5		WENO5	
	$\Delta t/\Delta x$	C_ε	$\Delta t/\Delta x^{5/3}$	C_ε	$\Delta t/\Delta x^{5/3}$	C_ε
Coastal Kelvin wave	5×10^{-5}	1	2.5×10^{-4}	1	2.5×10^{-4}	1000
Inertia–gravity wave	10^{-4}	1	10^{-3}	1	10^{-3}	500
Barotropic tide	2.5×10^{-4}	1	5×10^{-4}	1	5×10^{-4}	1
Manufactured solution	5×10^{-5}	10^{-4}	5×10^{-3}	1	5×10^{-3}	1
Stable barotropic jet	5×10^{-3}	1	5×10^{-1}	1	5×10^{-1}	1

where (u) is the forward Euler step:

$$\mathcal{F}(u) = \text{round}[u + \Delta t L(u), \varepsilon_{\text{TT}}^{u}]. \tag{28}$$

Note that the same procedure is applied for the time integration of other conserved variables. For all other rounding operations, we use the standard formula given in Eq. (24).

5. Numerical results

In this section, we demonstrate the performance of the TT finite-volume solver in a series of five numerical test cases. The datasets corresponding to all numerical examples are available in the online supplemental material. The first four of these cases are taken from the validation suite introduced in Bishnu et al. (2024); the coastal Kelvin wave, inertia–gravity wave, barotropic tide, and manufactured solution cases are all solved on a square computational domain $\Omega = [0, L] \times [0, L]$. For these cases, unless otherwise stated, we set $g = 10\ \text{m s}^{-2}$, $f = 10^{-4}\ \text{s}^{-1}$, $H = 1000\ \text{m}$, $c = \sqrt{gH} = 100\ \text{m s}^{-1}$, and $R = c/f = 10^6$ m as in Bishnu et al. (2024). For the fifth and final test case, we present a flow characterized by a stable barotropic jet, similar to that from Galewsky et al. (2004), adapted to the plane; we describe the relevant details in section 5e.

In all cases, the nondimensional forms of Eqs. (1)–(3) are solved (see appendixes A and B) using the third-order strong stability-preserving Runge–Kutta method (Shu and Osher 1988; Gottlieb et al. 2001). For the fifth-order reconstruction schemes, we set the time step Δt proportional to $\Delta x^{5/3}$ to maintain accuracy, and the proportionality constant is chosen such that the numerical solution remains stable for all time steps. Note that all time step sizes will be reported in terms of the nondimensional variables. In all TT simulations, Eqs. (24) and (25) are calculated using only the nondimensional variables, and the L_2 error is defined relative to the L_2 error of the same reconstruction method at the coarsest grid level. Both tensor-train and full-tensor simulations are performed on Apple M1 Max chip using MATLAB by ensuring a single-thread execution. To optimize the efficiency in MATLAB and avoid performance losses due to for loops, the full-tensor solver is implemented as suggested in Danis et al. (2025) to make use of MATLAB's vectorization. Finally, all surface integrals are computed using Gauss–Legendre quadrature rule in the TT format, as suggested in Alexandrov et al. (2023). See Table 1 for a summary of the numerical examples along with nondimensional time step to grid size ratios and C_ε in Eq. (25).

Finally, we should be careful to consider and discuss the differences between the results presented here for the SWEs and the results from Danis et al. (2025) for the compressible Euler equations. First, the SWE configurations considered here are chosen specifically to show the performance of our TT-FV methods as applied to problems relevant to large-scale geophysical flows, whereas the experiments presented in Danis et al. (2025) show that similar TT-FV methods can accurately resolve shocks and flow structures arising from turbulent motions, which are not present in hydrostatic models of the ocean and atmosphere. Additionally, as previously discussed in section 4b, the method used to compute the flux in the present case was chosen as it is more computationally efficient for the SWE configurations considered here. Finally, one can note that, in the best cases, the speedups reported in Danis et al. (2025) are an order of magnitude larger than the speedups reported here. In part, this could be because of how effectively the model state is able to be compressed by TT across different types of flows, but it is very likely that the biggest contribution to this difference in performance is the larger number of unknowns in the compressible Euler problems. The compressible Euler equations have five unknown quantities in a three-dimensional domain, while the SWEs have only three unknowns in a two-dimensional domain. It is well known that the effectiveness of TT compression increases with problem size, so it is natural that the larger problem sizes from Danis et al. (2025) would lead to more speedup. This is good news for the present case because as future work extends these TT-FV methods to more realistic configurations in layered, three-dimensional models, we expect the speedups presented here not just to generalize but to improve.

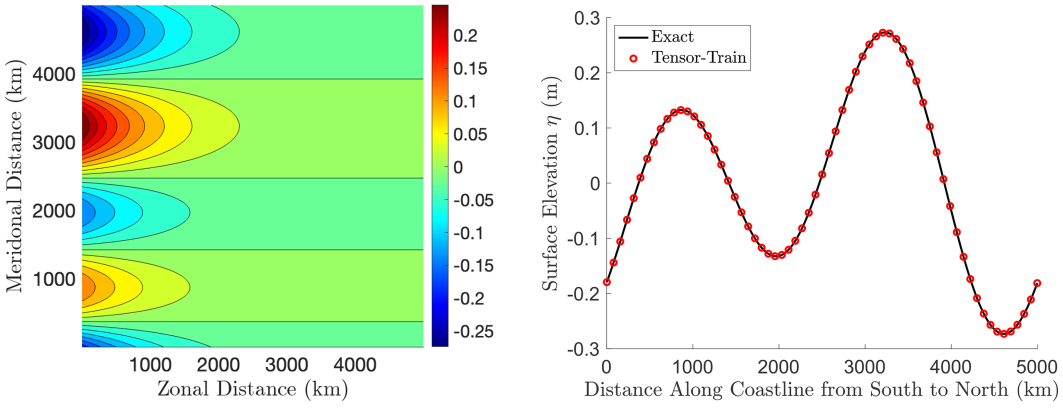
a. *Coastal Kelvin wave*

In this example, we solve the linear SWEs, Eqs. (1) and (2), up to the final time $T = 3$ h and set $L = 5 \times 10^6$ m. Coastal Kelvin waves occur when the Coriolis force is balanced against a topographic boundary and are found in real-world observations (Johnson and O'Brien 1990). In this idealized configuration, the coastline is in the nonperiodic x direction, and nondispersive waves travel along the boundary in the periodic y direction. The exact solution is plotted in Fig. 3, and the initial conditions are composed of two wave modes:

$$\begin{aligned}
\eta &= -H\{\hat{\eta}^{(1)} \sin[k_y^{(1)}(y + ct)] + \hat{\eta}^{(2)} \sin[k_y^{(1)}(y + ct)]\}\exp(-x/R),\\
u &= 0,\\
v &= \sqrt{gH}\{\hat{\eta}^{(1)} \sin[k_y^{(1)}(y + ct)] + \hat{\eta}^{(2)} \sin[k_y^{(1)}(y + ct)]\}\exp(-x/R),
\end{aligned} \tag{29}$$

where $\hat{\eta}^{(1)} = \hat{\eta}^{(2)}/2 = 10^{-4}$ m and $k_y^{(1)} = k_y^{(2)}/2 = 2\pi/L$.

Brought to you by Los Alamos National Laboratory | Unauthenticated | Downloaded 10/23/26 05:39 PM UTC



(a) Contour plot of η produced by the TT method (b) η profile

FIG. 3. Exact and TT solutions of surface elevation for coastal Kelvin wave at $T = 3$ h. The TT solution is obtained with Upwind5 on a 1280×1280 grid.

We use the exact solution to set the boundary condition in the x direction, but in the y direction, we consider a periodic boundary. Furthermore, for both upwind methods, we use $C_\varepsilon = 1$ in Eqs. (24) and (25) to calculate $\varepsilon_{\mathrm{TT}}$, but for WENO5, we use $C_\varepsilon = 1000$. To guarantee accuracy and stability of the numerical solution, we also consider a time step of $\Delta t/\Delta x = 5 \times 10^{-5}$ for the Upwind3 method and $\Delta t/\Delta x^{5/3} = 2.5 \times 10^{-4}$ for the fifth-order methods.

Figure 4 shows the performance of Upwind3, Upwind5, and WENO5 methods. On the left, all TT reconstruction methods are observed to maintain their formal L_2 convergence order for η. On the right, the various methods are compared in terms of the acceleration with respect to their corresponding standard full-tensor versions: At the finest grid level, TT Upwind5 achieves 83 times acceleration, while Upwind3 has a speedup of 73 times. Note that Upwind3 is still less expensive than Upwind5. Therefore, the higher speedup value of Upwind5 simply means that it becomes more efficient in accelerating its full-tensor version for this linear problem. On the other hand, the TT-WENO5 method only achieves the significantly lower speedup of 14 times. This clearly shows the cost of the TT cross interpolation used in the WENO5 scheme, even when the problem is linear and smooth.

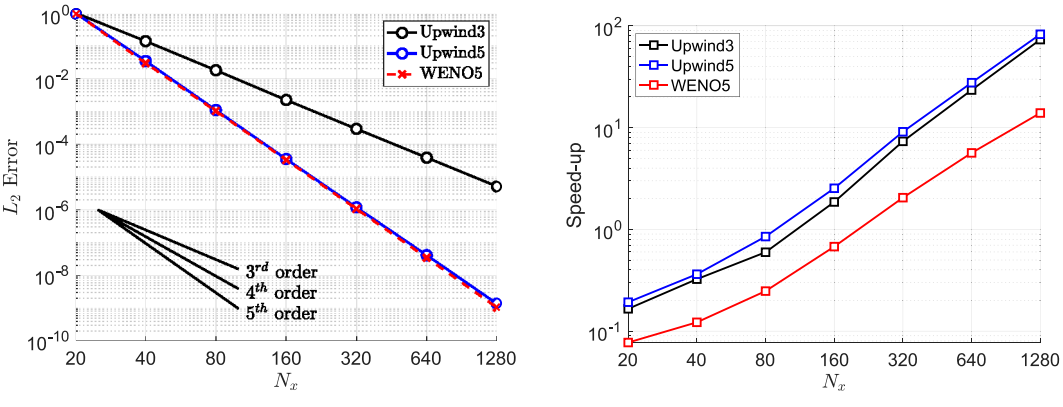
b. *Inertia–gravity wave*

Inertia–gravity waves are an important component of geophysical turbulence (Young 2021). They occur in the open ocean and are gravity waves where particles oscillate in an ellipse due to the rotation of Earth. The idealized test problem is linear with a flat bottom, like the coastal Kelvin wave, but the domain is periodic in both directions. Specifically, we solve the linear SWEs, Eqs. (1) and (2), up to the final time $T = 3$ h with $L = 10^7$ m. We consider the general solution

$$\eta = \hat{\eta}\cos(k_x x + k_y y - \hat{\omega} t),$$

$$u = \frac{g\hat{\eta}}{\hat{\omega}^2 - f^2}\{\hat{\omega}k_x \cos(k_x x + k_y y - \hat{\omega} t) - f k_y \sin(k_x x + k_y y - \hat{\omega} t)\},$$

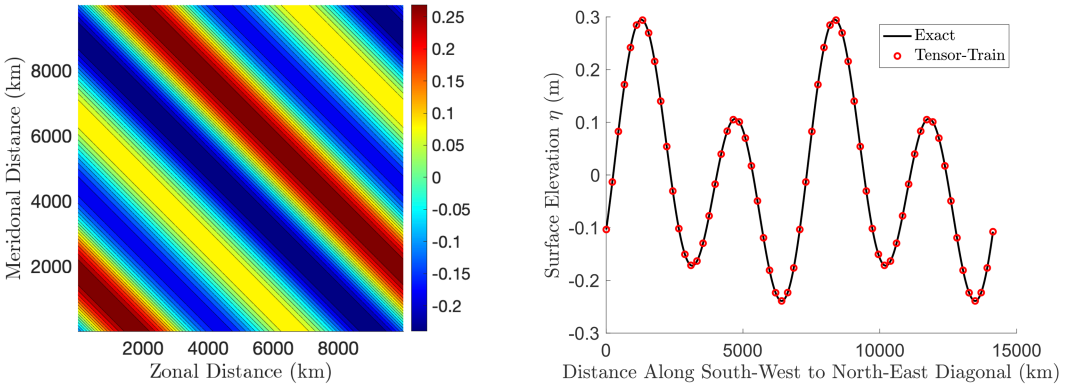
$$v = \frac{g\hat{\eta}}{\hat{\omega}^2 - f^2}\{\hat{\omega}k_y \cos(k_x x + k_y y - \hat{\omega} t) + f k_x \sin(k_x x + k_y y - \hat{\omega} t)\}, \tag{30}$$



(a) L_2 error for η (b) Speed-up

FIG. 4. The L_2 error of surface elevation and speedup for coastal Kelvin wave at $T = 3$ h.

Brought to you by Los Alamos National Laboratory | Unauthenticated | Downloaded 10/23/26 05:39 PM UTC



(a) Contour plot of η produced by the TT method (b) η profile

FIG. 5. Exact and TT solutions of surface elevation for inertia–gravity wave at $T = 3$ h. The TT solution is obtained with Upwind5 on a 1280×1280 grid.

where $\hat{\omega} = \sqrt{c^2(k_x^2 + k_y^2) + f^2}$. The particular solution plotted in Fig. 5 is constructed to be the superposition of two wave modes for $\eta_{(1)} = \eta^{(2)}/2 = 10^{-1}$ m, $k_x^{(1)} = k_x^{(2)}/2 = 2\pi/L$, and $k_y^{(1)} = k_y^{(2)}/2 = 2\pi/L$, and initial conditions are set from these at $t = 0$.

As in the previous example, Eqs. (24) and (25) use $C_\varepsilon = 1$ for both upwind methods, but they set $C_\varepsilon = 500$ for the WENO5 scheme in this example. Time step sizes are also set as $\Delta t/\Delta x = 10^{-4}$ for the Upwind3 method and $\Delta t/\Delta x^{5/3} = 10^{-3}$ for the fifth-order methods.

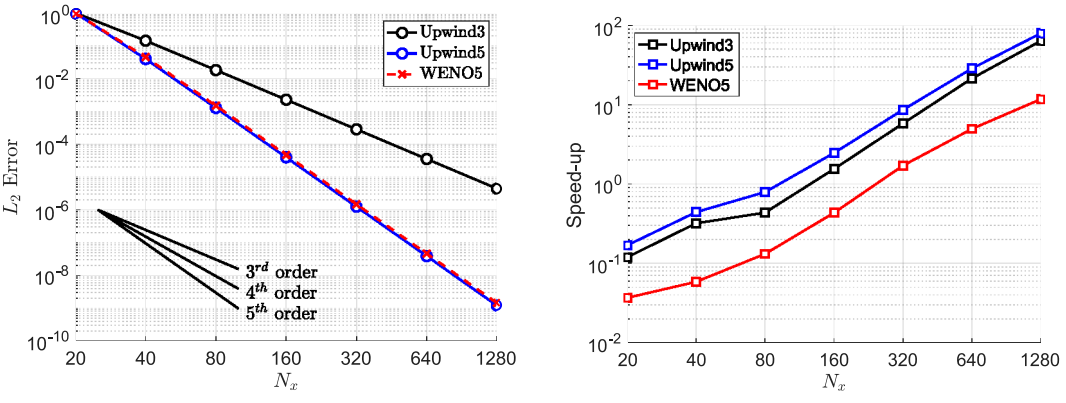
In Fig. 6, the L_2 convergence of η and speedup of the TT solver are shown. Our TT solver recovers the formal order of accuracy as in the previous example. Similar to the previous test case, TT Upwind5 results in a slightly better speedup than TT Upwind3, and TT WENO5 gives considerably less acceleration. At the finest grid level, the speedup achieved by the TT solver is 79 times for Upwind5, 64 times for Upwind3, and 12 times for WENO5.

c. *Barotropic tide*

The barotropic tide case is an idealized representation of a coastal tide on a continental shelf (Clarke and Battisti 1981). Like the previous cases, it tests the linear SWEs, Eqs. (1) and (2), but now on a doubly nonperiodic domain. We solve to a final time of $T = 30$ min, where $L = 25 \times 10^4$ m. The general solution considered here is given as

$$\eta = \hat{\eta}\cos(kx)\cos(\omega t),$$
$$u = \frac{g\hat{\eta}\omega k}{\hat{\omega}^2 - f^2}\sin(kx)\sin(\omega t),$$
$$v = \frac{g\hat{\eta}fk}{\hat{\omega}^2 - f^2}\sin(kx)\cos(\omega t), \tag{31}$$

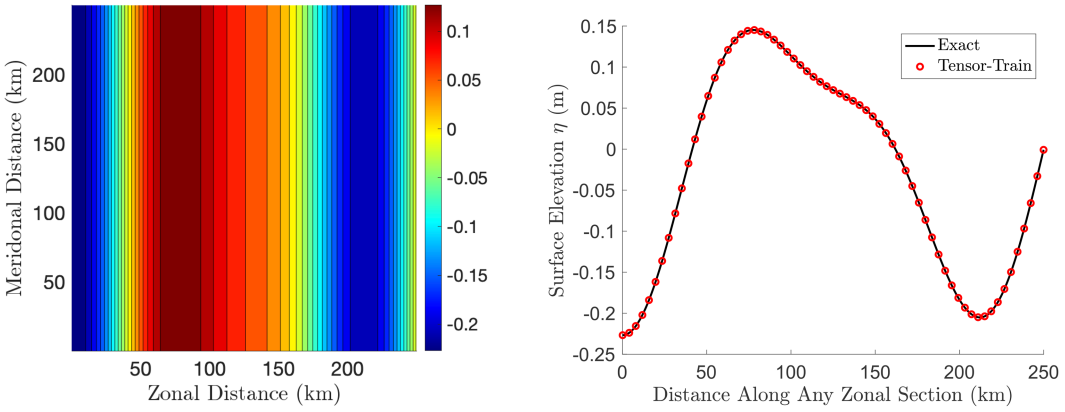
where $\omega = \sqrt{gHk^2 + f^2}$. We again consider a particular solution shown in Fig. 7 constructed as a superposition of two wave modes, but now, we set $\eta_{(1)} = \eta^{(2)}/2 = 0.2$ m, $k^{(1)} = 2\pi/\lambda^{(1)}$, and $k^{(2)} = 2\pi/\lambda^{(2)}$, where $\lambda^{(1)} = 4L/5$ and $\lambda^{(2)} = 4L/9$. Note that $H = 200$ m in this example. Physically, the wavelengths are chosen to satisfy the conditions for tidal resonance in this domain (Bishnu et al. 2024). The initial conditions at $t = 0$ are set from the exact solution, while the boundary condition in the x direction is taken from the exact solution and is assumed periodic in the y direction.



(a) L_2 error for η (b) Speed-up

FIG. 6. The L_2 error of surface elevation and speedup for inertia–gravity wave at $T = 3$ h.

Brought to you by Los Alamos National Laboratory | Unauthenticated | Downloaded 10/23/26 05:39 PM UTC



(a) Contour plot of η produced by the TT method (b) η profile

FIG. 7. Exact and TT solutions of surface elevation for barotropic tide at $T = 30$ min. The TT solution is obtained with Upwind5 on a 1280×1280 grid.

In this test case, we set $C_\varepsilon = 1$ for all reconstruction schemes and consider $\Delta t/\Delta x = 2.5 \times 10^{-4}$ for Upwind3 and $\Delta t/\Delta x^{5/3} = 5 \times 10^{-4}$ for the fifth-order methods.

The results reported in Fig. 8 follow the trends already observed in the previous linear test cases: Our TT solver achieves the formal order of accuracy, the highest speedup is obtained for Upwind5, and the WENO5 gives the lowest. At the finest grid level, the speedup achieved by the TT solver is 124 times for Upwind5, 89 times for Upwind3, and 19 times for WENO5.

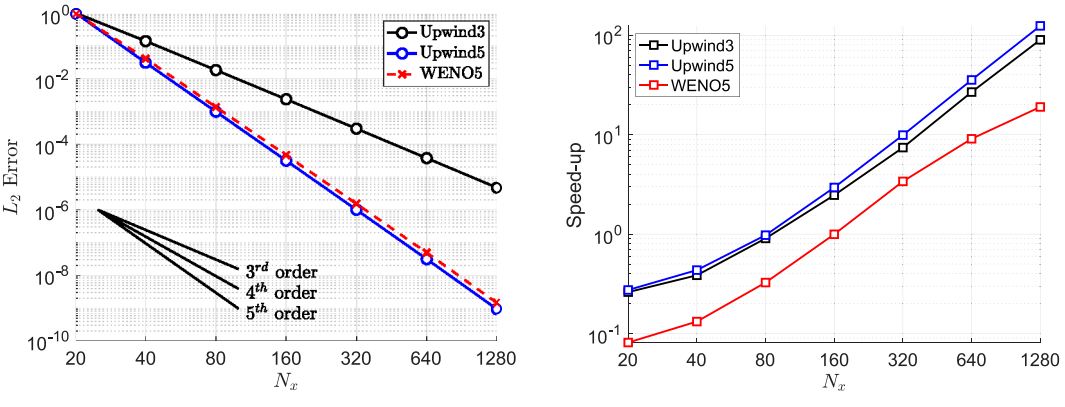
d. Manufactured solution

In this test case, we turn our attention to the nonlinear SWEs and solve Eqs. (1) and (3) up to the final time $T = 3$ h, where we set $L = 10^7$ m. The manufactured solution does not have an analog observed in nature but is important because an analytic solution may be derived that tests all terms in the SWEs. Subsequent SWE test cases for geophysical turbulence with more complex behavior do not have analytic solutions (Williamson et al. 1992). Using the method of manufactured solutions (Roache 2019), we enforce

$$\begin{aligned} \eta &= \hat{\eta}\sin(k_x x + k_y y - \hat{\omega}t), \\ u &= \hat{u}\cos(k_x x + k_y y - \hat{\omega}t), \\ v &= 0, \end{aligned} \tag{32}$$

where $\hat{\eta} = 10^{-2}$ m, $\hat{u} = 10^{-2}\,\mathrm{m\,s^{-1}}$, $k_x = k_y = 2\pi/L$, and $\hat{\omega} = \sqrt{c^2(k_x^2 + k_y^2)}$, as depicted in Fig. 9. We consider periodic boundaries and impose the initial condition from the manufactured solution at $t = 0$. In this nonlinear test case, we set $C_\varepsilon = 10^{-4}$ for the Upwind3 scheme to maintain its third-order accuracy. For the fifth-order methods, however, $C_\varepsilon = 1$ is deemed to be sufficient. The time stepping is performed according to $\Delta t/\Delta x = 5 \times 10^{-5}$ for Upwind3 and $\Delta t/\Delta x^{5/3} = 5 \times 10^{-3}$ for the fifth-order methods.

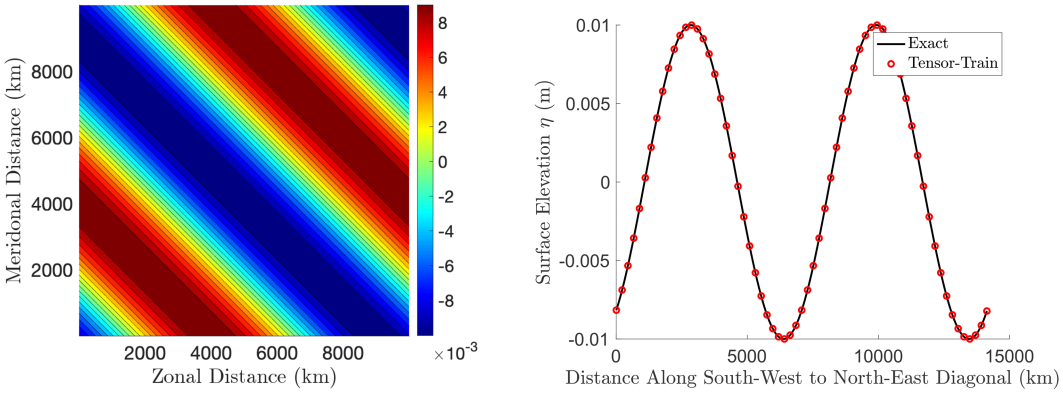
In Fig. 10, we observed the formal third-order L_2 convergence of Upwind3 and fifth-order convergence of Upwind5 and WENO5, as in the linear cases. However, we see that



(a) L_2 error for η (b) Speed-up

FIG. 8. The L_2 error of surface elevation and speedup for barotropic tide at $T = 30$ min.

Brought to you by Los Alamos National Laboratory | Unauthenticated | Downloaded 10/23/26 05:39 PM UTC



(a) Contour plot of η produced by the TT method (b) η profile

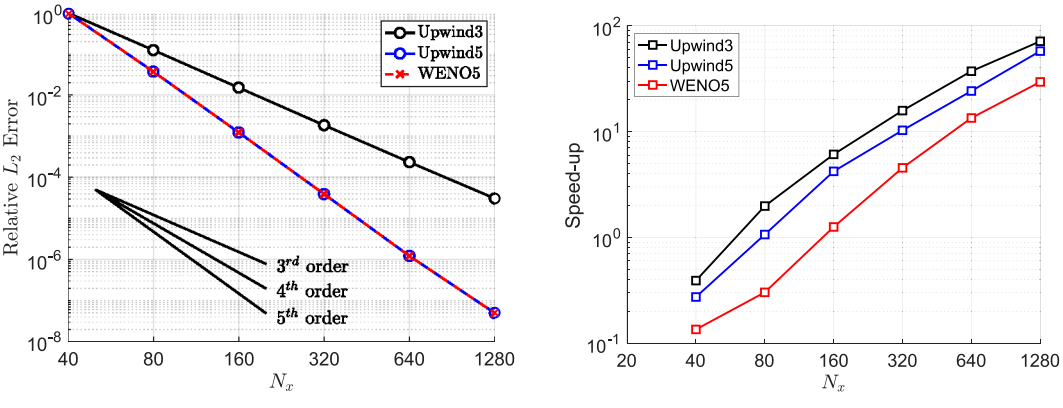
FIG. 9. Exact and TT solutions of surface elevation for the manufactured solution at $T = 3$ h. The TT solution is obtained with Upwind5 on a 1280×1280 grid.

Upwind3 gives a better acceleration than Upwind5 unlike the linear test cases. This may be due to the nonlinear nature of the fluxes in Eq. (3). Recall that TT rounding operation is performed after almost each TT multiplication and addition operation to avoid the rank growth. Therefore, TT rounding is applied several times for the nonlinear fluxes, while no rounding is needed for the linear fluxes. Since higher-order schemes require more quadrature points to compute fluxes, the additional cost of TT rounding is naturally more expensive for higher-order method for. In addition, the nonlinear fluxes need to compute $1/h_{\mathrm{TT}}$ using algorithm 1, which is also needed to be carried out on larger arrays for higher-order schemes. As a result, the different mechanics of flux computation in linear and nonlinear problems lead to different trends in terms of the speedup. At the finest grid level, the speedup achieved by the TT solver is 71 times for Upwind3, 57 times for Upwind5, and 29 times for WENO5.

e. *Stable barotropic jet*

In this final test case, we simulate a flow similar to that from the stable barotropic jet test case from Galewsky et al. (2004), adapted for the plane. We initialize the flow to a steady state characterized by a strong, balanced flow in the x direction. We run our model for 5 simulated days to show that it correctly maintains this steady state. In addition to showing that our TT solver produces flows equivalent to those produced by standard methods, we also show that the TT solver conserves thickness and energy within the expected error range. The unstable barotropic jet test case from Galewsky et al. (2004) will be included in future work, as the more complex dynamics will likely benefit from the further development of our TT methods.

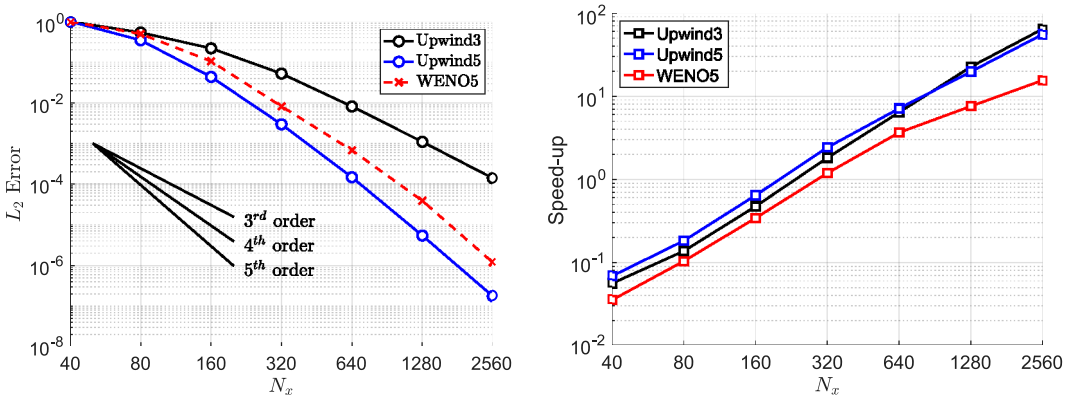
We solve the nonlinear SWEs [Eqs. (1) and (3)] with $g = 9.806\,16\ \mathrm{m\ s^{-2}}$, on the domain $\Omega = [0, L] \times [0, L/2]$, with $L = 2\pi \times 6\,371\,220$ m (the circumference of Earth). The computational domain is discretized by $N_x \times N_y$ grids, where $N_x = 5 \times 2^n$ for $n = 3, 4, \ldots, 9$ and $N_y = N_x/2$. The domain is periodic in x, with no-flow boundary conditions at $y = 0$ and $y = L/2$. To aid readability and implementation, we define two functions $\theta{:}[0, L] \to [0, 2\pi]$ and $\phi : [0, L/2] \to [-\pi/2, \pi/2]$ that map the x and y coordinates to domains normally associated with longitude and latitude:



(a) L_2 error for h (b) Speed-up

FIG. 10. The L_2 errors of the SWLT and speedup for the manufactured solution at $T = 3$ h.

Brought to you by Los Alamos National Laboratory | Unauthenticated | Downloaded 10/23/26 05:39 PM UTC



(a) L_2 error for h (b) Speed-up

FIG. 11. The L_2 errors of the SWLT and speedup for the stable barotropic jet case at $T = 5$ days.

$$\theta(x) = \frac{2\pi}{L}x,$$

$$\phi(y) = \frac{2\pi}{L}y - \frac{\pi}{2}. \tag{33}$$

The fluid velocity is initialized to so that the vertical component is equal to zero and the horizontal component depends only on y:

$$u(y) = \begin{cases} 0 & \phi(y) \le \phi_0, \\ C\exp\left\{\dfrac{1}{[\phi(y)-\phi_0][\phi(y)-\phi_1]}\right\} & \phi_0 < \phi(y) < \phi_1, \\ 0 & \phi(y) \ge \phi_1, \end{cases} \tag{34}$$

where $\phi_0 = -\pi/7$ and $\phi_1 = \pi/7$, $C = u_{\max}\exp[4/(\phi_1 - \phi_0)^2]$, and $u_{\max} = 80$ m s^{-1}. Then, a balanced initial condition for the thickness can be obtained by setting the time derivative of the velocity to zero, inserting Eq. (34) into the momentum equation, and solving for the thickness h. This gives us

$$h_{\text{balance}}(y) = h_0 - \frac{1}{g}\int_0^y fu(y')y', \tag{35}$$

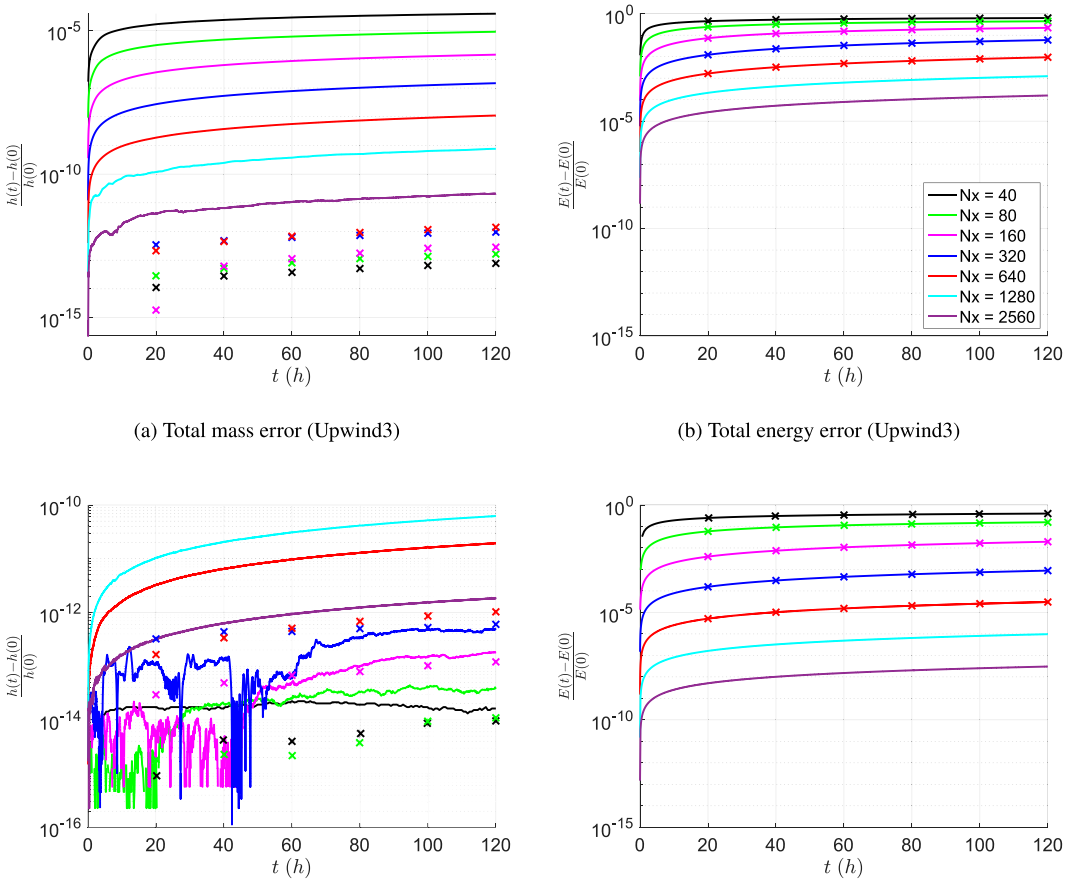
where h_0 is a constant chosen so that the average value of the thickness is equal to 10 000 m. For simplicity, we choose a constant value for the Coriolis force $f = 2\omega\sin(\pi/4)$, where $\omega = 7.292 \times 10^{-5}$ s^{-1} is the angular velocity of Earth's rotation.

In Fig. 11a, L_2 errors are measured by comparing the solution at $T = 5$ days to the exact solution taken from the initial conditions. All reconstruction schemes recover their respective formal convergence rate as the mesh is gradually refined. Unlike the previous cases, Upwind5 attains error values almost an order of magnitude smaller than those obtained by WENO5. This result is a clear outcome of smaller numerical dissipation levels of Upwind5 compared to WENO5, resulting in smaller modifications of the stable initial condition during the course of the simulation. Figure 11b shows the speedup values for all reconstruction schemes. The full-tensor simulations for the grid levels $n = 3, 4, \ldots, 7$ were run up to the final time $T = 5$ days. At the grid levels $n = 8$ and 9, the full-tensor simulation duration was measured for 100 times steps, which is then extrapolated to estimate the total computational cost at $T = 5$ days. Note that the tensor-train simulations were run up to the final time $T = 5$ days at all grid levels. As in the previous examples, the cross-interpolation procedure used for WENO5 results in lower-level accelerations. At the finest grid level, speedup values are 64.5 for Upwind3, 55.7 for Upwind5, and 15.6 for WENO5.

The global conservation properties of the proposed TT schemes are investigated in Fig. 12. Total mass is calculated as $h(t) = \int_\Omega h(t)A$, and total energy is calculated as $E(t) = \int_\Omega (1/2)[u(t)^2 + v(t)^2]A$ at any given time t. Tensor-train errors are denoted by solid lines, and full-tensor errors are denoted by symbols with the same color code for the same grid level. The vertical axis shows the difference between the total mass/energy at time $T = 0$ versus the total mass/energy at time $T = t$, referred to as the total mass/energy error. Finally, this error is normalized by dividing by the total energy at time $T = 0$. If these quantities were exactly conserved, the values on the vertical axis would be zero. This way of showing global conservation over time is similar to that used in Calandrini et al. (2021).

Figures 12a and 12c compare total mass errors of the tensor-train solver to those of the standard full-tensor solver for Upwind3 and Upwind5, respectively. Note that h is a conserved variable solved in Eq. (1). Therefore, the mismatch between the tensor-train and full-tensor error levels is a direct measure of applying TT rounding/truncation at the end of each Runge–Kutta stage, where additional numerical dissipation could be introduced. Despite this mismatch, however, the total mass errors for the TT solvers stabilize over time and tend to converge asymptotically to a steady-state value. On the other hand, it is interesting that the TT solver resulted in total energy errors that are almost identical to those obtained by the full-tensor solvers, as shown in Figs. 12b and 12d. In all cases, Upwind5 provides lower levels of total mass/energy errors, demonstrating the

Brought to you by Los Alamos National Laboratory | Unauthenticated | Downloaded 10/23/26 05:39 PM UTC



(a) Total mass error (Upwind3)

(b) Total energy error (Upwind3)

(c) Total mass error (Upwind5)

(d) Total energy error (Upwind5)

FIG. 12. Conservation of total mass and energy for the stable barotropic jet case. TT errors are denoted by solid lines, and full-tensor errors are denoted by symbols with the same color code for the same grid level.

advantage of using higher-order schemes where numerical dissipation is lower.

6. Conclusions

In this study, we developed a high-order tensor-train finite-volume solver for linear and nonlinear shallow-water equations (SWEs). The implementation of the third-order Upwind3, fifth-order Upwind5, and fifth-order WENO5 reconstruction schemes in the tensor-train format were presented in detail. We demonstrated that all reconstruction schemes achieved their formal order of convergence in the TT format for the linear and nonlinear SWEs. The upwind methods perform reconstruction procedures that directly modify TT cores, while the WENO scheme uses TT cross interpolation, due to which the WENO scheme was the method with lowest speedup values among all numerical results. For the linear SWEs, the fluxes are computed directly without using any TT rounding or cross interpolation, which resulted in speedup values up to 124 times compared to the standard full-tensor implementation. However, the nonlinear SWEs need to calculate the reciprocal of the shallow water layer thickness (SWLT) in the TT format followed by applying TT rounding several times, which was reflected in our numerical results by reducing the speedup values compared to the linear case. To compute the TT reciprocal of the SWLT, we employed an approximation based on Taylor series, which turned out to be quite efficient since the SWLT can usually be decomposed as a large mean value and small oscillations. Overall, we showed that the TT finite-volume method maintains the accuracy of the underlying numerical discretization while significantly accelerating it for the SWEs.

The long-term goal of future work will be to adapt TT methods to layered Boussinesq models of the ocean and atmosphere at the climate scale. In the short term, there are two open questions of particular interest. First, we wish to extend these methods to spherical, unstructured grids like those employed by the ocean and atmosphere components of the U.S. Department of Energy's Energy Exascale Earth System Model (E3SM) (Golaz et al. 2022); it is not clear how the complex geometry of these more sophisticated grids will affect the compression obtained by the TT methods as we additionally move toward more complex flows. The compression of the model state by TT methods is the primary source of speedup in this study, so it needs to be understood how exactly this

Brought to you by Los Alamos National Laboratory | Unauthenticated | Downloaded 10/23/26 05:39 PM UTC

compression is affected by increasing the complexity of the problem. Eventually, we hope to apply TT methods to three-dimensional ocean and atmosphere domains; we are confident that this will provided increased speedup versus standard methods as the benefit of TT compression increases with problem size and dimension. Second, as these methods are advanced toward climate applications, we need to compare performance gains against leadership-class climate codes. This will require that we effectively leverage high-performance computing (HPC) hardware and libraries so that a direct comparison can be made. We expect that moving toward HPC architectures will reveal additional speedups, as much of the currently emerging HPC hardware specifically targets the types of tensor operations that are used by TT methods. The results obtained here and in similar works on TT methods suggest that these methods could precipitate a paradigm shift in how we model geophysical fluids.

Acknowledgments. The authors gratefully acknowledge the support of the Laboratory Directed Research and Development (LDRD) program of Los Alamos National Laboratory under project 20230067DR and 20240782ER and ISTI Rapid Response 20248215CT-IST. Los Alamos National Laboratory is operated by Triad National Security, LLC, for the National Nuclear Security Administration of U.S. Department of Energy (Contract 89233218CNA000001). The authors thank Siddhartha Bishnu and Joseph Schoonover for useful discussions about the test cases.

Data availability statement. The data that support the findings of this research are available from the corresponding author upon reasonable request.

APPENDIX A

High-Order Reconstructions for Upwind and WENO Methods

Here, we present the details necessary to implement the Upwind3, Upwind5, and WENO5 reconstruction schemes.

a. Step 1: decell averaging over x

First, we discuss the reconstruction of $\tilde{u}^{-}_{i+(1/2),j}$, which is a one-dimensional cell average in y. The procedure to reconstruct $\tilde{u}^{+}_{i-(1/2),j}$ is similar (see Shu 1998); therefore, it will not be presented here. First, we set $v_{i,j} = \tilde{\tilde{u}}_{i,j}$, but the index j is dropped in $v_{i,j}$ below for simplicity:

$$\tilde{u}^{-}_{i+(1/2),j} = \sum_{r=0}^{k} \omega_r v^{(r)}_{i+1/2}. \tag{A1}$$

For the third-order upwind method (Upwind3), $k = 1$ and

$$\begin{aligned} v^{(0)}_{i+1/2} &= \frac{1}{2}(v_i + v_{i+1}),\\ v^{(1)}_{i+1/2} &= \frac{1}{2}(-v_{i-1} + 3v_i), \end{aligned} \tag{A2}$$

while for the fifth-order Upwind5 and WENO5 schemes, $k = 2$ and

$$\begin{aligned} v^{(0)}_{i+1/2} &= \frac{1}{6}(2v_i + 5v_{i+1} - v_{i+2}),\\ v^{(1)}_{i+1/2} &= \frac{1}{6}(-v_{i-1} + 5v_i + 2v_{i+1}),\\ v^{(2)}_{i+1/2} &= \frac{1}{6}(2v_{i-2} - 7v_{i-1} + 11v_i). \end{aligned} \tag{A3}$$

In the upwind methods, ω_r are determined from the ideal linear weights. For example, $\omega_0 = 2/3$ and $\omega_1 = 1/3$ for Upwind3, while $\omega_0 = 3/10$, $\omega_1 = 3/5$, and $\omega_2 = 1/10$ for Upwind5. For the WENO5 reconstruction, we compute the nonlinear weights using

$$\omega_r = \frac{\alpha_r}{\sum_{s=0}^{2} \alpha_s}, \tag{A4}$$

where $\alpha_r = d_r/(\beta_r + \varepsilon)^2$ for $r = 0, 1, 2$, $\varepsilon = \Delta x^2$ as suggested by Don and Borges (2013), $d_0 = 3/10$, $d_1 = 3/5$, and $d_2 = 1/10$ (same as the ideal linear weights), and the smoothness indicators are given as

$$\begin{aligned} \beta_0 &= \frac{13}{12}(v_i - 2v_{i+1} + v_{i+2})^2 + \frac{1}{4}(3v_i - 4v_{i+1} + v_{i+2})^2,\\ \beta_1 &= \frac{13}{12}(v_{i-1} - 2v_i + v_{i+1})^2 + \frac{1}{4}(v_{i-1} - v_{i+1})^2,\\ \beta_2 &= \frac{13}{12}(v_{i-2} - 2v_{i-1} + v_i)^2 + \frac{1}{4}(v_{i-2} - v_{i-1} + 3v_i)^2. \end{aligned} \tag{A5}$$

b. Step 2: decell averaging over y

Now that we have one-dimensional cell averages in y, we will next complete the de-cell averaging by reconstructing pointwise values at $x_{i\pm1/2}$ and $y_j + \delta_m \Delta y$ for each quadrature point m. The procedure is similar to step 1, but linear and nonlinear reconstruction weights are specifically defined for each quadrature point m.

As in step 1, we will set $v_{i+1/2,j} = \tilde{u}^{-}_{i+(1/2),j}$ but drop the index $i + 1/2$ in $v_{i+1/2,j}$ for simplicity. Then,

$$u^{-}_{i+(1/2),y_j+\delta_m\Delta y} = \sum_{r=0}^{k} \omega^{(r)}_m v^{(r)}_m, \tag{A6}$$

where

$$v^{(r)}_m = \sum_{l=0}^{k} c^m_{rl} v_{j-r+l}, \tag{A7}$$

and $\{c_{rl}\}^m$ denotes the components of the coefficient matrix $\mathbf{C}^m$ for the particular quadrature point m.

For the Upwind3 scheme where $k = 1$, we perform the reconstruction procedure on two quadrature points. The coefficient matrix for each quadrature point is given as

Brought to you by Los Alamos National Laboratory | Unauthenticated | Downloaded 10/23/26 05:39 PM UTC

$$\mathbf{C}^{m=1} = \begin{pmatrix} 808/627 & -390/1351 \\ 390/1351 & 961/1351 \end{pmatrix},$$

$$\mathbf{C}^{m=2} = \begin{pmatrix} 961/1351 & 390/1351 \\ -390/1351 & 808/627 \end{pmatrix}, \tag{A8}$$

and the linear weights are $\omega_m^{(r)} = 1/2$ for $r = 0, 1$ and $m = 1, 2$.

For the Upwind5 and WENO5 schemes where $k = 2$, we consider a reconstruction on three quadrature points. The coefficient matrices for these points are given as

$$\mathbf{C}^{m=1} = \begin{pmatrix} 4725/2927 & -2051/2438 & 249/1097 \\ 249/1097 & 14/15 & -467/2913 \\ -467/2913 & 366/517 & 2209/4883 \end{pmatrix},$$

$$\mathbf{C}^{m=2} = \begin{pmatrix} 23/24 & 1/12 & -1/24 \\ -1/24 & 13/12 & -1/24 \\ -1/24 & 1/12 & 23/24 \end{pmatrix},$$

$$\mathbf{C}^{m=3} = \begin{pmatrix} 2209/4883 & 366/517 & -467/2913 \\ -467/2913 & 14/15 & 249/1097 \\ 249/1097 & -2051/2438 & 4725/2927 \end{pmatrix}. \tag{A9}$$

For the fifth-order schemes with three quadrature points along the cell interfaces, the ideal linear coefficients of the second quadrature point, $m = 2$, become negative, and these are treated according to the method presented in Shi et al. (2002). Following their method, we first set the linear weights for the first and the last quadrature point as

$$\gamma_{m=1}^{(r=0)} = 882/6305, \quad \gamma_{m=1}^{(r=1)} = 403/655, \quad \gamma_{m=1}^{(r=2)} = 463/1891,$$
$$\gamma_{m=3}^{(r=0)} = 463/1891, \quad \gamma_{m=3}^{(r=1)} = 403/655, \quad \gamma_{m=3}^{(r=2)} = 882/6305.$$

Then, the split coefficients are used for the second quadrature point:

$$\gamma_{m=2}^{(r=0)+} = 9/214, \quad \gamma_{m=2}^{(r=1)+} = 98/107, \quad \gamma_{m=2}^{(r=2)+} = 9/214,$$
$$\gamma_{m=2}^{(r=0)-} = 9/67, \quad \gamma_{m=2}^{(r=1)-} = 49/67, \quad \gamma_{m=2}^{(r=2)-} = 9/67.$$

The Upwind5 scheme simply sets $\omega_m^{(r)} = \gamma_m^{(r)}$ for $m = 1, 3$ and $r = 0, 1, 2$ as these coefficients are already positive and then applies Eq. (A6). For the second quadrature point $m = 2$, we perform the reconstruction using

$$u^{-}_{i+(1/2),y_j+\delta_2\Delta y} = \sigma^{+}u^{+} - \sigma^{-}u^{-}, \tag{A10}$$

where $\sigma^{+} = 107/40$ and $\sigma^{-} = 67/40$ with

$$u^{\pm} = \sum_{r=0}^{k} \gamma_{m=2}^{(r)\pm} v_{m=2}^{(r)}. \tag{A11}$$

The WENO5 scheme is similar to Upwind5, but it computes the nonlinear weights, instead. For $m = 1, 3$, it uses Eq. (A6) with the nonlinear weights:

$$\omega_m^{(r)} = \frac{\alpha_m^{(r)}}{\sum_{s=0}^{2} \alpha_m^{(s)}}, \tag{A12}$$

where $\alpha_m^{(r)} = \gamma_m^{(r)}/(\beta_r + \varepsilon)^2$ for $r = 0, 1, 2$, and the smoothness indicators are given as

$$\beta_0 = \frac{13}{12}(v_j - 2v_{j+1} + v_{j+2})^2 + \frac{1}{4}(3v_j - 4v_{j+1} + v_{j+2})^2,$$
$$\beta_1 = \frac{13}{12}(v_{j-1} - 2v_j + v_{j+1})^2 + \frac{1}{4}(v_{j-1} - v_{j+1})^2,$$
$$\beta_2 = \frac{13}{12}(v_{j-2} - 2v_{j-1} + v_j)^2 + \frac{1}{4}(v_{j-2} - 4v_{j-1} + 3v_j)^2. \tag{A13}$$

For the second quadrature point $m = 2$, WENO5 uses Eq. (A10) with the same $\sigma^{\pm}$, but it sets $u^{\pm}$ as

$$u^{\pm} = \sum_{r=0}^{k} \omega_{m=2}^{(r)\pm} v_{m=2}^{(r)}, \tag{A14}$$

where $\alpha_2^{(r)\pm} = \gamma_{m=2}^{(r)\pm}/(\beta_r + \varepsilon)^2$ for $r = 0, 1, 2$.

For the reconstruction procedure in the y direction, the abovementioned procedures are repeated by applying step 1 in the y direction to construct $\overline{u}_{i,j+1/2}$ (a one-dimensional cell average in x) and step 2 in the x direction to compute $u^{\pm}_{x_i+\delta_m\Delta x, y_{j+1/2}}$ on quadrature points m.

APPENDIX B

Nondimensional Form of the Shallow-Water Equations

Here, we will briefly describe the "nondimensionalization" of the governing equations. Denoting dimensional variables by a star, let us rewrite the shallow-water equations:

$$\frac{\partial \mathbf{U}^*}{\partial t^*} + \frac{\partial \mathbf{F}^*}{\partial x^*} + \frac{\partial \mathbf{G}^*}{\partial y^*} = \mathbf{S}^*. \tag{B1}$$

For the linear case, we have

$$\mathbf{U}^* = \begin{pmatrix} \eta^* \\ u^* \\ v^* \end{pmatrix}, \quad \mathbf{F}^* = \begin{pmatrix} H^* u^* \\ g^* \eta^* \\ 0 \end{pmatrix}, \quad \mathbf{G}^* = \begin{pmatrix} H^* v^* \\ 0 \\ g^* \eta^* \end{pmatrix}, \quad \mathbf{S}^* = \begin{pmatrix} 0 \\ f^* v^* \\ -f^* u^* \end{pmatrix}, \tag{B2}$$

and for the nonlinear case, we have

$$\mathbf{U}^* = \begin{pmatrix} h^* \\ h^* u^* \\ h^* v^* \end{pmatrix}, \quad \mathbf{F}^* = \begin{pmatrix} h^* u^* \\ h^* u^{*2} + \frac{1}{2} g^* h^{*2} \\ h^* u^* v^* \end{pmatrix},$$
$$\mathbf{F}^* = \begin{pmatrix} h^* v^* \\ h^* u^* v^* \\ h^* v^{*2} + \frac{1}{2} g^* h^{*2} \end{pmatrix}, \quad \mathbf{S}^* = \begin{pmatrix} 0 \\ f^* h^* v^* \\ -f^* h^* u^* \end{pmatrix}. \tag{B3}$$

Next, we define characteristic scales for each variable in the SWEs, such as the reference length L^*_{ref}, reference velocity U^*_{ref}, reference time $t^*_{\text{ref}} = L^*_{\text{ref}}/U^*_{\text{ref}}$, reference height H^*_{ref}, reference acceleration $g^*_{\text{ref}} = U^{*2}_{\text{ref}}/H^*_{\text{ref}}$, and reference frequency $f^*_{\text{ref}} = U^*_{\text{ref}}/L^*_{\text{ref}}$. Then, the nondimensional parameters are defined as

Brought to you by Los Alamos National Laboratory | Unauthenticated | Downloaded 10/23/26 05:39 PM UTC

TABLE B1. Characteristic scales for numerical examples.

Test case	L^*_{ref}(m)	U^*_{ref}(m s^{-1})	H^*_{ref}(m)
Coastal Kelvin wave	5×10^6	5×10^{-3}	0.1
Inertia–gravity wave	10^7	1.622×10^{-3}	0.2
Barotropic tide	25×10^4	3.163×10^{-3}	0.2
Manufactured solution	10^7	10^{-2}	500
Stable barotropic jet	$2\pi \times 6\,371\,220$	20	8000

$$
\begin{aligned}
t &= \frac{t^*}{t^*_{\text{ref}}}, \quad x = \frac{x^*}{L^*_{\text{ref}}}, \quad y = \frac{y^*}{L^*_{\text{ref}}},\\
\eta &= \frac{\eta^*}{H^*_{\text{ref}}}, \quad h = \frac{h^*}{H^*_{\text{ref}}},\\
u &= \frac{u^*}{U^*_{\text{ref}}}, \quad v = \frac{v^*}{U^*_{\text{ref}}},\\
g &= \frac{g^*}{g^*_{\text{ref}}}, \quad f = \frac{f^*}{f^*_{\text{ref}}}.
\end{aligned} \tag{B4}
$$

Substituting these into Eqs. (B1)–(B3) gives Eqs. (1)–(3). Note that both set of equations have the same form. However, in the dimensional form, C_ε in Eqs. (24) and (25) needs to attain extremely small values, e.g., $C_\varepsilon = 10^{-18}$–10^{-22}, due to extremely large computational domains. On the other hand, the nondimensional form significantly decreases this ambiguity, and $C_\varepsilon = 1$ seems to work well in the most numerical examples discussed in section 5.

Finally, we summarize the reference values used in the nondimensionalization of the governing equations in Table B1.

REFERENCES

Alexandrov, B., G.Manzini, E. W.Skau, P. M. D.Truong, and R. G.Vuchov, 2023: Challenging the curse of dimensionality in multidimensional numerical integration by using a low-rank tensor-train format. *Mathematics*, **11**, 534, https://doi.org/10.3390/math11030534.

Archibald, R., K. J.Evans, J.Drake, and J. B.WhiteIII, 2011: Multiwavelet discontinuous Galerkin-accelerated exact linear part (ELP) method for the shallow-water equations on the cubed sphere. *Mon. Wea. Rev.*, **139**, 457–473, https://doi.org/10.1175/2010MWR3271.1.

Bachmayr, M., 2023: Low-rank tensor methods for partial differential equations. *Acta Numer.*, **32**, 1–121, https://doi.org/10.1017/S0962492922000125.

Bishnu, S., M. R.Petersen, B.Quaife, and J.Schoonover, 2024: A verification suite of test cases for the barotropic solver of ocean models. *J. Adv. Model. Earth Syst.*, **16**, e2022MS003545, https://doi.org/10.1029/2022MS003545.

Calandrini, S., D.Engwirda, and M.Petersen, 2021: Comparing numerical accuracy and stability for different horizontal discretizations in MPAS-Ocean. *Ocean Modell.*, **168**, 101908, https://doi.org/10.1016/j.ocemod.2021.101908.

Caldwell, P. M., and Coauthors, 2019: The DOE E3SM coupled model version 1: Description and results at high resolution. *J. Adv. Model. Earth Syst.*, **11**, 4095–4146, https://doi.org/10.1029/2019MS001870.

Cichocki, A., 2014: Tensor networks for big data analytics and large-scale optimization problems. arXiv, 1407.3124v2, https://doi.org/10.48550/arXiv.1407.3124.

Clarke, A. J., and D. S.Battisti, 1981: The effect of continental shelves on tides. *Deep-Sea Res.*, **28A**, 665–682, https://doi.org/10.1016/0198-0149(81)90128-X.

Cushman-Roisin, B., and J.-M.Beckers, 2011: *Introduction to Geophysical Fluid Dynamics: Physical and Numerical Aspects.*Academic Press, 828 pp.

Danis, M. E., and B.Alexandrov, 2023: Navier-Stokes: Tensorization of the Sod and Blowoff problems. LANL Tech. Rep. LA-UR-24-24283, 51 pp., https://www.osti.gov/biblio/2346026.

——, D.Truong, I.Boureima, O.Korobkin, K. Ø.Rasmussen, and B. S.Alexandrov, 2025: Tensor-train WENO scheme for compressible flows. *J. Comput. Phys.*, **529**, 113891, https://doi.org/10.1016/j.jcp.2025.113891.

Demir, S., R.Johnson, B. T.Bojko, A. T.Corrigan, D. A.Kessler, P. Guthrey, J.Burmark, and S.Irving, 2024: Three-dimensional compressible chemically reacting computational fluid dynamics with tensor trains. *AIAA SCITECH 2024 Forum*, Orlando, FL, AIAA, AIAA 2024-1337, https://doi.org/10.2514/6.2024-1337.

Dennis, J., A.Fournier, W. F.Spotz, A.St-Cyr, M. A.Taylor, S. J. Thomas, and H.Tufo, 2005: High-resolution mesh convergence properties and parallel efficiency of a spectral element atmospheric dynamical core. *Int. J. High Perform. Comput. Appl.*, **19**, 225–235, https://doi.org/10.1177/1094342005056108.

Dolgov, S. V., and D. V.Savostyanov, 2014: Alternating minimal energy methods for linear systems in higher dimensions. *SIAM J. Sci. Comput.*, **36**, A2248–A2271, https://doi.org/10.1137/140953289.

Don, W.-S., and R.Borges, 2013: Accuracy of the weighted essentially non-oscillatory conservative finite difference schemes. *J. Comput. Phys.*, **250**, 347–372, https://doi.org/10.1016/j.jcp.2013.05.018.

Donahue, A. S., and Coauthors, 2024: To exascale and beyond—The Simple Cloud-Resolving E3SM Atmosphere Model (SCREAM), a performance portable global atmosphere model for cloud-resolving scales. *J. Adv. Model. Earth Syst.*, **16**, e2024MS004314, https://doi.org/10.1029/2024MS004314.

Galewsky, J., R. K.Scott, and L. M.Polvani, 2004: An initial-value problem for testing numerical models of the global shallow-water equations. *Tellus*, **56A**, 429–440, https://doi.org/10.3402/tellusa.v56i5.14436.

Golaz, J.-C., and Coauthors, 2022: The DOE E3SM model version 2: Overview of the physical model and initial model evaluation. *J. Adv. Model. Earth Syst.*, **14**, e2022MS003156, https://doi.org/10.1029/2022MS003156.

Goreinov, S. A., I. V.Oseledets, D. V.Savostyanov, E. E. Tyrtyshnikov, and N. L.Zamarashkin, 2010: How to find a good submatrix. *Matrix Methods: Theory, Algorithms and Applications*, World Scientific, 247–256.

Gottlieb, S., C.-W.Shu, and E.Tadmor, 2001: Strong stability-preserving high-order time discretization methods. *SIAM Rev.*, **43**, 89–112, https://doi.org/10.1137/S003614450036757X.

Gourianov, N., 2022: Exploiting the structure of turbulence with tensor networks. Ph.D thesis, University of Oxford, 122 pp.

Johnson, M. A., and J. J.O'Brien, 1990: The role of coastal Kelvin waves on the northeast Pacific Ocean. *J. Mar. Syst.*, **1**, 29–38, https://doi.org/10.1016/0924-7963(90)90085-O.

Kay, J. E., and Coauthors, 2015: The Community Earth System Model (CESM) large ensemble project: A community resource for studying climate change in the presence of internal climate variability. *Bull. Amer. Meteor. Soc.*, **96**, 1333–1349, https://doi.org/10.1175/BAMS-D-13-00255.1.

Kiehl, J. T., J. J.Hack, G. B.Bonan, B. A.Boville, D. L.Williamson, and P. J.Rasch, 1998: The National Center for Atmospheric

Brought to you by Los Alamos National Laboratory | Unauthenticated | Downloaded 10/23/26 05:39 PM UTC

Research Community Climate Model: CCM3. *J. Climate*, **11**, 1131–1149, https://doi.org/10.1175/1520-0442(1998)011<1131:TNCFAR>2.0.CO;2.

Kiffner, M., and D.Jaksch, 2023: Tensor network reduced order models for wall-bounded flows. *Phys. Rev. Fluids*, **8**, 124101, https://doi.org/10.1103/PhysRevFluids.8.124101.

Lilly, J. R., D.Engwirda, G.Capodaglio, R. L.Higdon, and M. R. Petersen, 2023: CFL optimized forward–backward Runge–Kutta schemes for the shallow-water equations. *Mon. Wea. Rev.*, **151**, 3191–3208, https://doi.org/10.1175/MWR-D-23-0113.1.

Mahoney, M. W., and P.Drineas, 2009: Cur matrix decompositions for improved data analysis. *Proc. Natl. Acad. Sci. USA*, **106**, 697–702, https://doi.org/10.1073/pnas.0803205106.

Oseledets, I., 2011: Tensor-train decomposition. *SIAM J. Sci. Comput.*, **33**, 2295–2317, https://doi.org/10.1137/090752286.

——, 2014: oseledets/TT-Toolbox. GitHub, accessed 1 June 2024, https://github.com/oseledets/TT-Toolbox.

——, and E.Tyrtyshnikov, 2010: TT-cross approximation for multidimensional arrays. *Linear Algebra Appl.*, **432**, 70–88, https://doi.org/10.1016/j.laa.2009.07.024.

Petersen, M. R., D. W.Jacobsen, T. D.Ringler, M. W.Hecht, and M. E.Maltrud, 2015: Evaluation of the arbitrary Lagrangian–Eulerian vertical coordinate method in the MPAS-ocean model. *Ocean Modell.*, **86**, 93–113, https://doi.org/10.1016/j.ocemod.2014.12.004.

Roache, P. J., 2019: The method of manufactured solutions for code verification. *Computer Simulation Validation. Simulation Foundations, Methods and Applications*, C.Beisbart and N.Saam, Eds., Springer, 295–318, https://doi.org/10.1007/978-3-319-70766-2_12.

Savostyanov, D., and I.Oseledets, 2011: Fast adaptive interpolation of multi-dimensional arrays in tensor train format. *The 2011 Int. Workshop on Multidimensional (nD) Systems*, Poitiers, France, IEEE, 1–8, https://doi.org/10.1109/nDS.2011.6076873.

Shi, J., C.Hu, and C.-W.Shu, 2002: A technique of treating negative weights in WENO schemes. *J. Comput. Phys.*, **175**, 108–127, https://doi.org/10.1006/jcph.2001.6892.

Shu, C.-W., 1998: Essentially non-oscillatory and weighted essentially non-oscillatory schemes for hyperbolic conservation laws. *Advanced Numerical Approximation of Nonlinear Hyperbolic Equations*, A.Quarteroni, Ed., Lecture Notes in Mathematics, Vol. 1697, Springer, 325–432, https://doi.org/10.1007/BFb0096355.

——, and S.Osher, 1988: Efficient implementation of essentially non-oscillatory shock-capturing schemes. *J. Comput. Phys.*, **77**, 439–471, https://doi.org/10.1016/0021-9991(88)90177-5.

Thuburn, J., T. D.Ringler, W. C.Skamarock, and J. B.Klemp, 2009: Numerical representation of geostrophic modes on arbitrarily structured C-grids. *J. Comput. Phys.*, **228**, 8321–8335, https://doi.org/10.1016/j.jcp.2009.08.006.

Truong, D. P., M. I.Ortega, I.Boureima, G.Manzini, K. Ø.Rasmussen, and B. S.Alexandrov, 2024: Tensor networks for solving the time-independent Boltzmann neutron transport equation. *J. Comput. Phys.*, **507**, 112943, https://doi.org/10.1016/j.jcp.2024.112943.

von Larcher, T., and R.Klein, 2019: On identification of self-similar characteristics using the Tensor Train decomposition method with application to channel turbulence flow. *Theor. Comput. Fluid Dyn.*, **33**, 141–159, https://doi.org/10.1007/s00162-019-00485-z.

Weller, H., H. G.Weller, and A.Fournier, 2009: Voronoi, delaunay, and block-structured mesh refinement for solution of the shallow-water equations on the sphere. *Mon. Wea. Rev.*, **137**, 4208–4224, https://doi.org/10.1175/2009MWR2917.1.

Williamson, D. L., J. B.Drake, J. J.Hack, R.Jakob, and P. N. Swarztrauber, 1992: A standard test set for numerical approximations to the shallow water equations in spherical geometry. *J. Comput. Phys.*, **102**, 211–224, https://doi.org/10.1016/S0021-9991(05)80016-6.

Young, W. R., 2021: Inertia-gravity waves and geostrophic turbulence. *J. Fluid Mech.*, **920**, F1, https://doi.org/10.1017/jfm.2021.334.

Brought to you by Los Alamos National Laboratory | Unauthenticated | Downloaded 10/23/26 05:39 PM UTC